

Síntesis, traducción y verificación de sistemas de ecuaciones utilizados en Protocolos de Conocimiento Cero

Lucía Martínez Martínez

Grado en Ingeniería Informática
Universidad Complutense de Madrid



Trabajo de Fin de Grado
Curso 2025 - 2026

Directores:

Clara Rodríguez Núñez
Alejandro Hernández Cerezo

Synthesis, translation and verification of constraints systems in the context of Zero Knowledge Protocols

Lucía Martínez Martínez

Degree in Computer Science
Complutense University of Madrid



Final-year project
Academic Year 2025–2026

Directors:
Clara Rodríguez Núñez
Alejandro Hernández Cerezo

Resumen

Este trabajo presenta la implementación de una nueva funcionalidad para el compilador de Circom, diseñada para simplificar la creación de circuitos mediante una traducción automática de operadores de alto nivel a restricciones algebraicas. La complejidad de este lenguaje radica en la necesidad de definir manualmente un sistema de ecuaciones en formato R1CS coherente y seguro, ya que una implementación incompleta o errónea de estas restricciones puede comprometer la integridad del circuito, permitiendo la generación de pruebas falsas.

Con el objetivo de resolver esta problemática, se han especificado reglas que permiten transformar operaciones complejas (como comparaciones, divisiones, expresiones condicionales, etc.) en ecuaciones válidas para una prueba de conocimiento cero, garantizando que el witness generado sea siempre fiel a la lógica imperativa del programador.

Finalmente, la validez de estas reglas ha sido sometida a un proceso de verificación mediante la herramienta ZK-GENVER, asegurando que las restricciones resultantes sean robustas y seguras frente a posibles vulnerabilidades, para luego ser aplicadas sobre circuitos reales, incluyendo implementaciones de juegos simples.

Palabras clave: Circom, compilador, witness, prueba de conocimiento cero, sistemas de ecuaciones R1CS, circuitos aritméticos.

Abstract

This work presents the implementation of a new functionality for the Circom compiler, designed to simplify circuit creation through the automatic translation of high-level operators into algebraic constraints. The complexity of this language lies in the need to manually define a coherent and secure system of equations in R1CS format, since an incomplete or erroneous implementation of these constraints can compromise the integrity of the circuit, allowing the generation of false proofs.

With the aim of resolving this issue, rules have been specified that allow the transformation of complex operations (such as comparisons, divisions, conditional expressions, etc.) into valid equations for a zero-knowledge proof, guaranteeing that the generated witness is always faithful to the programmer's imperative logic.

Finally, the validity of these rules has been subjected to a verification process using the ZK-GENVER tool, ensuring that the resulting constraints are robust and secure against potential vulnerabilities, to then be applied to real-world circuits, including simple game implementations.

Keywords: Circom, compilers, witness computation, zero-knowledge proof, R1CS equation systems, arithmetic circuits.

Agradecimientos

A Clara y Alejandro, por la oportunidad de realizar este trabajo con ellos y por introducirme en el mundo de Circom. Un área que era completamente desconocida para mí y que hoy en día se ha convertido en uno de los campos de la informática que más despiertan mi interés.

También quiero agradecerles su atención, paciencia y dedicación. Un apoyo que, en el caso de Clara, viene de mucho antes. Su ayuda en múltiples asignaturas a lo largo de la carrera ha sido fundamental para afrontar las asignaturas más difíciles y evitar que todo se volviese cuesta arriba.

A la Universidad Complutense de Madrid, por permitirme cursar mis estudios aquí y formarme en lo que más me ha gustado desde pequeña, la informática. Una etapa que me ha brindado la mejor educación posible y un gran crecimiento personal.

A la educación pública, ya que sin ella nunca habría podido tener acceso a una carrera universitaria, y mucho menos en Madrid.

Al IES Melchor de Macanaz, lugar donde cursé mis estudios obligatorios y bachillerato, conociendo a grandes profesores. Entre ellos, quiero agradecer especialmente a Dani y Pilar, ya que sin ellos no habría nacido mi interés en las matemáticas y la física, las cuales despertaron la curiosidad que me hizo decantarme finalmente por la informática.

A mi familia. Sin ellos no habría podido tener la posibilidad de poder estudiar una carrera, y mucho menos en Madrid, apoyándome tanto económicamente como emocionalmente, siendo un pilar fundamental durante mis estudios.

En especial a mi madre, Gloria, por escucharme pacientemente en todo lo que le contaba sobre la carrera, aunque no entendiese nada, y por apoyarme de forma incondicional, llegando a firmar el alquiler de un piso en Madrid incluso antes de que hiciera la prueba de acceso a la universidad.

A mis abuelos, Lourdes y Juanjo, por interesarse siempre por cómo me iba en Madrid cada vez que regresaba, esperándome con los brazos abiertos. A mi abuela, por sus tupperes de comida, sobre todo los de carrilleras. Y a mi abuelo, por esas ganas de que llegara para tomarnos juntos un trago de su vino (según él, aguachirri).

A mi tía Rosana, por estar siempre ahí, especialmente durante un primer curso que se me hizo cuesta arriba. Su cercanía, apoyo y ayuda me dieron el empujón definitivo para recordarme de lo que era capaz.

A Ana, por ser mi mayor apoyo en Madrid y la persona que me enseñó a disfrutar de la ciudad. Por recordarme que no todo es estudiar y que la vida también va de exprimir y disfrutar cada momento.

A Bonita, por ser la perra que más ha impedido el desarrollo de este trabajo por ponerse encima del teclado para llamar mi atención.

A mis amigas del pueblo, que aunque ya no estén, fueron una gran parte de esta etapa, guardando grandes recuerdos.

Pero sobre todo, a mi pueblo, Hellín. Al acabar bachillerato lo único que pensaba era en irme de allí y empezar una nueva vida en una gran ciudad. A día de hoy, solo echo la mirada atrás, pensando en volver.

Índice general

1. Introducción a Circom	8
1.1. Zero Knowledge Proof	8
1.2. CIRCOM	9
1.3. R1CS	11
1.4. Snarkjs	13
1.5. Preámbulo del problema	13
1.6. Objetivos y Estructura del trabajo	15
1.6.1. Objetivos	15
1.6.2. Estructura del trabajo	15
2. Introduction to Circom	17
2.1. Zero-Knowledge Proofs	17
2.2. CIRCOM	18
2.3. R1CS	20
2.4. Snarkjs	22
2.5. Problem Preamble	22
2.6. Objectives and Work Structure	24
2.6.1. Objectives	24
2.6.2. Work Structure	24
3. Definición del problema	26
3.1. Descripción del problema	26
3.2. Ejemplo: Piedra, Papel y Tijera	28
3.2.1. Señales de los jugadores	29
3.2.2. Señal del ganador	30
3.2.3. Lógica del empate	32
3.2.4. Lógica de los jugadores	33
4. Desarrollo del problema	35
4.1. Síntesis de restricciones para la traducción de expresiones	35
4.1.1. Casos inductivos: Reglas de transformación de expresiones	36
4.1.1.1. Operadores Aritméticos	36
4.1.1.2. Operadores Lógicos	41
4.1.1.3. Operadores de Comparación	43
4.1.1.4. Operadores de desplazamiento	51
4.1.2. Casos inductivos: Reglas de transformación de instrucciones	52
4.1.2.1. Instrucciones condicionales	52

5. Implementación del problema	58
5.1. Ejemplo de implementación	59
6. Verificación de los operadores	63
6.1. Resultados obtenidos	66
6.1.1. Operadores Aritméticos	66
6.1.2. Operadores Lógicos	67
6.1.3. Operadores de Comparación	67
6.1.4. Operadores de Desplazamiento	68
7. Experimentos	69
7.1. Aplicación de la generación automática sobre juegos simples	69
7.1.1. Piedra, Papel y Tijera	69
7.1.2. Caníbales y misioneros	70
7.1.3. Tres en raya	72
7.1.4. Sudoku	73
7.2. Aplicación de la generación automática sobre circomlib y comparación de constraints	75
8. Conclusiones y Trabajo Futuro	77
8.1. Conclusiones	77
8.2. Trabajo Futuro	78
9. Conclusions and Future Work	79
9.1. Conclusions	79
9.2. Future Work	80
A. Problemática de los números negativos	83
B. Verificación de operadores	84
B.1. Operadores Aritméticos	84
B.1.1. División	84
B.1.2. Módulo	85
B.2. Operadores lógicos	85
B.2.1. AND binario	85
B.2.2. OR binario	86
B.3. Operadores de desplazamiento	87
B.3.1. Desplazamiento a la izquierda	87
B.3.2. Desplazamiento a la derecha	88

Introducción a Circom

1.1. Zero Knowledge Proof

En términos criptográficos, una prueba de conocimiento cero, conocida como Zero Knowledge Proof (o sus siglas inglesas ZKP), es una **familia de protocolos** que permite el establecimiento de métodos cuyo objetivo es permitir que una parte (**prover**) demuestre a otra (**verifier**) la validez de una afirmación **sin revelar información sobre la misma**, más allá de su validez [1].

A continuación se presenta un ejemplo para ilustrar este concepto:

Alejandro (prover) tiene el décimo de la lotería de Navidad que ha sido premiado y quiere demostrarle a **Clara (verifier)** que posee el número ganador, pero sin que ésta sea conocedora del mismo (y así evitar que pueda cobrar el premio o se lo robe). Para ello, utilizan una caja fuerte que solo se abre introduciendo la combinación del decimo premiado.

Clara escribe en un papel un dígito aleatorio, lo introduce en la caja y la cierra. Posteriormente, Clara se gira, de forma que no ve nada más de la caja.

Alejandro, que conoce el número del décimo, introduce la combinación, abriendo así la caja y leyendo el número escrito por Clara en el papel en voz alta. En ese momento, Clara sabe que ha podido abrir la caja, y por tanto, obtiene una prueba de que Alejandro es conocedor de esa información.

Sin embargo, Clara podría sospechar que Alejandro ha adivinado el dígito al azar, ya que la probabilidad es del 10 %, por lo que Clara decide repetir la prueba varias veces.

Sí repetimos la prueba una segunda vez ($0,1 * 0,1 = 0,01$), se reduce la probabilidad inicial de 0,1 a 0,01. Es decir, a medida que las repeticiones aumentan, menor es la probabilidad de que se acierte al azar, ya que ésta converge a 0, alcanzando así una certeza práctica sin haber revelado ni un solo dígito del décimo.

Ejemplo 1.1: Zero Knowledge Proof



Como hemos visto en el *Ejemplo 1.1*, el concepto de **ZKP** queda ilustrado de una

forma muy intuitiva. Sin embargo, para trasladar esta lógica al contexto criptográfico, es necesario formalizar estas interacciones mediante protocolos específicos.

La **criptografía** constituye el pilar fundamental de la seguridad en el mundo digital, permitiendo proteger información sensible frente a amenazas. Las técnicas criptográficas nos ofrecen la capacidad de garantizar confidencialidad así como un equilibrio entre transparencia y privacidad.

Históricamente, el término de ZKP apareció en los años 80 como un concepto puramente teórico, hasta su desarrollo en la práctica [2]. Dicha noción surgió con la intención de reforzar tanto la seguridad como la privacidad en el ámbito criptográfico. Actualmente, esto ha llevado a la creación de grandes familias de protocolos, destacando especialmente **zk-SNARKs**.

Definición 1 (zk-SNARKS) *Succinct Non-Interactive Argument of Knowledge, mayormente conocido por sus siglas **zk-SNARKs**, son pruebas criptográficas las cuales permiten probar la veracidad de información sin revelar el contenido de la misma. [3].*

Estas pruebas destacan por su reducido tamaño de prueba y corto tiempo de verificación. Sin embargo, presentan la desventaja de requerir una fase inicial conocida como **configuración de confianza**.

Definición 2 (Configuración de confianza) *Una configuración de confianza es una fase en la que se produce la generación de valores aleatorios que deben destruirse de forma inmediata. Si estos valores aleatorios son expuestos, se compromete la seguridad del sistema [3].*

El motivo de que estos número aleatorios deban borrarse inmediatamente es debido a que cualquier entidad conocedora de esos valores podría almacenarlos y así generar **pruebas falsas**, de forma que se podría vulnerar la seguridad del sistema.

Dada la complejidad que implica implementar estos protocolos desde cero, se han desarrollado lenguajes de programación específicos, también conocidos como **Domain Specific Languages** o DSL.

El objetivo es que los desarrolladores que no son expertos en criptografía puedan programar las declaraciones que desean demostrar. Es en este contexto donde cobra importancia **Circom**, una herramienta creada con el fin de facilitar la creación de circuitos y su posterior transformación en sistemas de restricciones.

1.2. CIRCOM

Circom es un lenguaje de programación basado en la descripción de circuitos (DSL), cuyo objetivo principal no es el cálculo, sino definir un sistema de ecuaciones que describe las relaciones entre las **señales** de entrada y salida [4].

Una señal es un componente que actúa como cable dentro del circuito, el cual transporta valores a través de las puertas lógicas del mismo y que forma parte de las ecuaciones que el programador desea verificar. Las restricciones del sistema de ecuaciones generado por Circom describen el comportamiento de las señales del circuito.

Esta visión es fundamental, ya que las ZKP no pueden interpretar código directamente; requieren que la computación se exprese en una estructura de restricciones algebraicas denominadas R1CS, que detallaremos en la **Sección 1.3**.

El proceso de compilación de Circom refleja un paradigma híbrido entre lo **declarativo e imperativo**. De esta forma, el compilador genera dos archivos esenciales para generar la prueba:

- **Archivo R1CS:** Contiene el conjunto de todas las restricciones algebraicas que definen la lógica del circuito. Estas restricciones son añadidas desde un programa Circom por medio de los símbolos `<==` y `===`.
- **Generador de Testigo (Witness):** Un ejecutable (en formato C++ o WASM) encargado de calcular los valores de todas las señales del circuito, incluyendo los valores intermedios, a partir de las entradas proporcionadas. Estas asignaciones son añadidas desde un programa Circom por medio de los símbolos `<==` y `<--`.

Partiendo de estos dos archivos, podemos generar la prueba por medio de **Snarkjs**. Esta herramienta toma el archivo R1CS y el Witness obtenidos tras la compilación de un programa Circom para generar la prueba asociada. Analizaremos en detalle su funcionamiento y comandos específicos en la **Sección 1.4**.

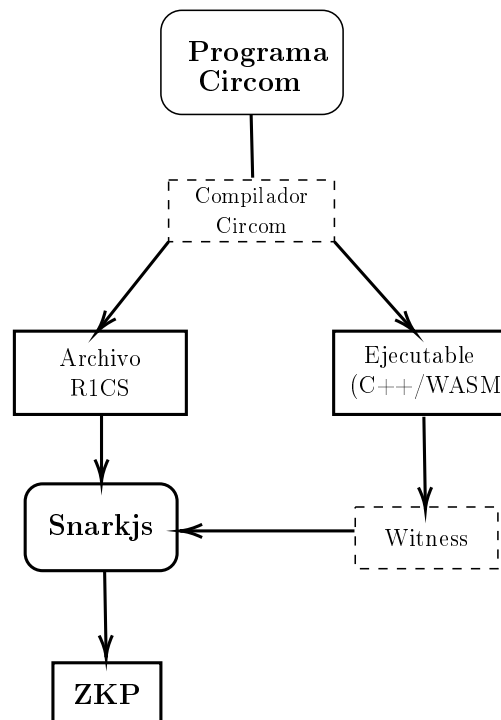


Figura 1.1: Proceso de transformación a ZKP desde Circom

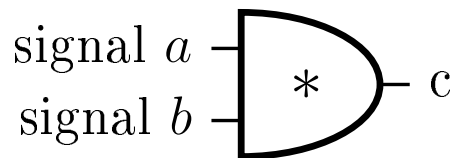
Una característica fundamental de Circom es que es un lenguaje basado en circuitos, donde la estructura del mismo debe ser estática. Esto implica que cualquier valor determinante en la estructura del circuito (como tamaños de arrays, número de iteraciones de un bucle, etc..) **debe conocerse en tiempo de compilación**. Esto se debe a que el archivo R1CS contiene las restricciones algebraicas que describen el circuito, las cuales no pueden depender de los valores de las señales de entrada.

Como hemos mencionado anteriormente, las señales son las restricciones establecidas por el programador, las cuáles quiere verificar. A continuación se muestra un ejemplo de un programa Circom y su representación en un circuito lógico, en el que se añade la restricción $a * b = c$.

```

1      template mult(){
2          signal input a;
3          signal input b;
4          signal output c;
5          c <== a * b;
6      }
7
8      main component = mult();

```



Ejemplo 1.2: Ilustración comparativa código-circuito

■

El ejemplo anterior refleja como Circom transforma la ecuación $c <== a * b$ en un circuito lógico equivalente. De esta forma podemos ver que las entradas del componente lógico son a y b , forzando de esta manera a que c sea igual al producto de ambas entradas.

1.3. R1CS

Rank-1 Constraint System (conocido como **R1CS**), es un sistema de representación matemático diseñado para transformar tareas computacionales complejas en un conjunto de restricciones algebraicas. Su relevancia incide en la propiedad de que generar la solución a un problema puede ser caro, mientras que su **verificación es muy eficiente y rápida**, clave en las pruebas de conocimiento cero [5].

Para que este sistema de representación sea funcional, las restricciones de R1CS se definen sobre un cuerpo finito $\mathbb{F}_p$.

Definición 3 (Cuerpo finito) *En el álgebra abstracta, un cuerpo finito es una estructura algebraica que contiene un número finito de elementos. Esto significa*

que las operaciones aplicadas sobre estos elementos se realizan usando aritmética modular en un rango $[0, p-1]$, con p siendo un número primo que determina el orden del cuerpo [6].

Circom suele utilizar primos muy grandes para estos cuerpos finitos. Por ejemplo, uno de los primos que Circom está utilizando actualmente correspondo con:

$p = 52435875175126190479447740508185965837690552500527637822603658699938581184513$

R1CS impone restricciones matemáticas complejas. Las ecuaciones, aparte de no poder exceder el segundo grado (*condición necesaria, pero no suficiente*), tienen que poder expresarse como el producto de dos combinaciones lineales de variables, teniendo como resultado una tercera combinación lineal:

$$A \cdot B - C = 0$$

Donde A , B y C son combinaciones lineales de las señales del circuito. En términos prácticos, esto implica que una sola restricción solo puede contener una multiplicación entre variables. Operaciones que incluyan múltiples productos independientes, aunque sean de segundo grado, deben ser descompuestas en varias restricciones consecutivas por medio de variables auxiliares con el fin de cumplir con este estándar.

Siguiendo el *Ejemplo 1.2*, la restricción $c \leq a * b$ se traduce al formato R1CS como el producto de dos combinaciones lineales A y B , forzando así que el valor de la suma de las señales a y b sea c .

Ejemplo 1.3: Transformación de multiplicación formato R1CS

Dada la declaración:

$$(a) \times (b) = (c)$$

Donde la estructura $A \cdot B = C$ se satisface asignando:

- $A = a$ (Combinación lineal 1)
- $B = b$ (Combinación lineal 2)
- $C = c$ (Resultado de la restricción)

■

Gracias a utilizar el formato R1CS, las restricciones escritas en Circom se pueden verificar de forma sumamente eficiente a través de Snarkjs, como veremos en la **Sección 1.4**.

1.4. Snarkjs

Snarkjs es una librería de JavaScript creada para la generación y verificación de pruebas de conocimiento cero partiendo de las restricciones en formato R1CS correspondientes, que puede ser ejecutado tanto en dispositivos móviles como en servidores con *Node.js*, materializando el concepto de *zk-SNARKS* nombrado anteriormente [4][7].

Como se mostró previamente en la *Figura 1.1*, Snarkjs requiere de dos archivos para generar la prueba:

- *Witness*, generado por el ejecutable C++/WASM.
- *Archivo R1CS*, restricciones de la lógica del circuito.

Una vez disponibles estos dos archivos y la configuración de confianza finalizada, Snarkjs genera la prueba de conocimiento cero que permite que el Prover pueda demostrarle al Verifier la validez de su declaración.

1.5. Preámbulo del problema

Circom es un lenguaje de programación cuyo aprendizaje y aplicación puede ser complejo inicialmente, sobre todo si se carece de habilidad tratando con lenguajes lógicos basados en restricciones. Una de las razones principales de esta complejidad son los conocidos como *underconstrained bugs*.

Definición 4 (Underconstrained bug) *Un underconstrained bug es un error producido cuando el usuario no introduce todas las ecuaciones necesarias en el fichero R1CS para reflejar el comportamiento del circuito.*

Un programa Circom es correcto cuando las ecuaciones algebraicas del archivo R1CS representan exactamente el comportamiento del circuito (expresado de forma imperativa en el generador de witness). La ausencia de las restricciones conlleva que pueden generarse witnesses que no reflejan el correcto comportamiento del circuito, pero que se aceptarían al verificar las ecuaciones del R1CS.

El símbolo `<--` permite añadir una asignación al archivo Witness, pero no al R1CS, ya que este es exclusivo para las restricciones del programa (añadidas con los operadores `<==` y `==`).

La funcionalidad de estos operadores es completamente diferente. Al principio, muchas personas que comienzan a programar en Circom no tienen clara la diferencia real entre estos operadores, desarrollando programas erróneos cuya prueba generada no es válida.

Un uso incorrecto de estos operadores podría provocar problemas como el mencionado underconstrained bug, de forma que si el usuario utiliza `<--` para añadir restricciones al sistema de ecuaciones en lugar de `<==` o `==`, habrá ecuaciones que

no serán añadidas al archivo R1CS, y por tanto, no formarán parte del sistema de ecuaciones que describe el circuito, quedando incompleto.

Un ejemplo de este tipo de error es el siguiente bloque de código, cuyo objetivo es el de comprobar si una entrada es igual a cero:

```
1   template isZero() {
2       signal input a;
3       signal output b;
4
5       signal inv;
6
7       inv <-- a!= 0 ? 1/a : 0;
8
9       b <== -a*inv * 1;
10  }
```

Código 1.1: Underconstrained bug

Esta función, en apariencia correctamente implementada, presenta un mal uso del operador `<--`, provocando el mencionado underconstrained bug. Al utilizar este operador para el cálculo de la señal `inv`, se realiza una asignación pero no se añade una restricción al sistema de ecuaciones.

De esta forma, al no haber ninguna restricción que obligue a que `inv` sea el inverso de `a`, el vínculo entre la entrada `a` con la salida `b` no se cumple totalmente. Por tanto, al no ser añadida la asignación `inv <-- a!= 0 ? 1/a : 0` al sistema de ecuaciones, no existe para el verificador, permitiendo que un prover malicioso pueda manipular el valor de `inv` para generar pruebas falsas que el sistema aceptaría.

Más allá del ejemplo expuesto, este error se repite de forma recurrente en muchos programas Circom, sobre todo aquellos desarrollados por principiantes en el lenguaje. Otros ejemplos donde este error está presente son en implementaciones de terceros que pueden encontrarse en repositorios públicos como el juego de "*Hundir la flota*" [8], *Dark Forest* [9] o el *Decoder template en circomlib* [10].

Debido a este error, entre muchos otros, Circom es un lenguaje complejo, al que cuesta adaptarse, en gran parte por los diferentes conceptos a similar. Por ello, el objetivo de este trabajo es facilitar al usuario una forma de programar en Circom muchas más rápida y sencilla, así como segura.

Aprovechando la flexibilidad de las asignaciones y la sintaxis ofrecida por la naturaleza *declarativa-imperativa* de Circom, podemos implementar una nueva funcionalidad realizando modificaciones en su compilador, permitiendo así traducir la lógica imperativa (restricciones declaradas con `<--`, únicamente añadidas al *Witness*) en restricciones que formarán parte del sistema de ecuaciones (únicamente añadidas por medio de `<==`).

De este modo, **el compilador actuaría como un puente**, transformando automáticamente el flujo imperativo en un sistema de restricciones.

Dicha transformación se consigue mediante la traducción de instrucciones imperativas de Circom. Este proceso consiste en analizar los operadores admitidos por el lenguaje Circom y especificar una regla de transformación para cada uno de ellos. De esta forma, el compilador consigue traducir automáticamente lo implementado a un sistema de restricciones.

1.6. Objetivos y Estructura del trabajo

1.6.1. Objetivos

El objetivo principal de este trabajo es crear una manera más sencilla e intuitiva de programar en Circom, consiguiendo que la creación de sistemas de ecuaciones y sus circuitos lógicos equivalentes sea mucho más accesible y segura.

De manera complementaria, el trabajo busca obtener una herramienta de validación. Aprovechando las nuevas capacidades del compilador, permitir al usuario comparar su implementación en Circom con la generada mediante la nueva funcionalidad, facilitando la comprobación entre su lógica y la generada por el compilador.

Este último objetivo también incluye el aprendizaje, buscando reducir la complejidad inicial de Circom, permitiendo al usuario familiarizarse con el lenguaje de una forma más sencilla y eficiente.

Para alcanzar los objetivos propuestos, hemos realizado las siguientes tareas durante el desarrollo del trabajo:

1. Diseño y especificación de las reglas de traducción
2. Implementación dentro del compilador de las reglas especificadas
3. Verificación de los sistemas de ecuaciones generados por las reglas
4. Aplicación de la nueva funcionalidad en casos reales
5. Desarrollo de la memoria para recoger todo lo anterior y analizar los resultados del trabajo

1.6.2. Estructura del trabajo

Para alcanzar los objetivos propuestos, este documento se ha estructurado en los siguientes capítulos:

- Capítulo 1 (Introducción a Circom): Se introduce el contexto sobre la tecnología que vamos a tratar y ofrecer una visión general de lo que pretendemos hacer con ella a lo largo del trabajo.
- Capítulo 2 (Definición del problema): Se definirá el problema y se presentará un ejemplo.
- Capítulo 3 (Desarrollo del problema): Especificación técnica de las modificaciones que se llevarán a cabo en el compilador del lenguaje.

- Capítulo 4 (Implementación): Detalle del desarrollo de las nuevas funcionalidades en el código fuente del compilador.
- Capítulo 5 (Verificación de operadores): Análisis del proceso de comprobación para asegurar que los nuevos operadores generen sistemas de restricciones correctos y equivalentes.
- Capítulo 6 (Experimentos): Evaluación del rendimiento y comportamiento del compilador modificado frente a implementaciones manuales de juegos simples y comparación de los nuevos operadores con los de `circomlib`.
- Capítulo 7 (Conclusiones y Trabajo Futuro): Recapitulación de los resultados obtenidos, valoración del impacto de la solución y propuestas para futuras investigaciones.

Introduction to Circom

2.1. Zero-Knowledge Proofs

In cryptographic terms, a zero-knowledge proof (ZKP) refers to a **family of protocols** designed to enable one party (**the prover**) to demonstrate to another (**the verifier**) that a specific statement is true **without revealing any information about it** other than its validity [1].

The following example illustrates this concept:

Alejandro (prover) holds a winning Christmas lottery ticket and wants to prove to **Clara (verifier)** that he owns the winning number without her actually seeing it (to prevent her from claiming the prize or stealing it). To do this, they use a safe that only opens by entering the winning ticket's combination.

Clara writes a random digit on a piece of paper, places it inside the safe, and locks it. Then, Clara turns around so she can no longer see the safe.

Alejandro, who knows the ticket number, enters the combination to open the safe and reads the number Clara wrote out loud. At that moment, Clara knows he was able to open the safe and, therefore, obtains proof that Alejandro possesses that information.

However, Clara might suspect that Alejandro simply guessed the digit, as there is a 10 % chance of doing so. Consequently, Clara decides to repeat the test several times.

If we repeat the test a second time ($0,1 * 0,1 = 0,01$), the initial probability drops from 0.1 to 0.01. In other words, as the number of repetitions increases, the probability of guessing correctly by chance decreases and converges to 0, achieving practical certainty without revealing a single digit of the ticket.

Ejemplo 2.1: Zero Knowledge Proof



As shown in *Example 2.1*, the concept of **ZKP** is illustrated in a very intuitive way. However, to translate this logic into a cryptographic context, these interactions must be formalized through specific protocols.

Cryptography is the fundamental pillar of digital security, protecting sensitive

information from threats. Cryptographic techniques allow us to guarantee confidentiality while maintaining a balance between transparency and privacy.

Historically, the term ZKP emerged in the 1980s as a purely theoretical concept before its practical development [2]. This notion was created to strengthen both security and privacy within the cryptographic field. Today, this has led to the creation of major protocol families, most notably **zk-SNARKs**.

Definition 1 (zk-SNARKS) *Succinct Non-Interactive Argument of Knowledge, better known by the acronym **zk-SNARKs**, are cryptographic proofs that allow one to prove the truth of information without revealing its content [?].*

These proofs are notable for their small proof size and short verification time. However, they have the disadvantage of requiring an initial phase known as a **trusted setup**.

Definition 2 (Trusted setup) *A trusted setup is a phase where random values are generated and must be destroyed immediately. If these random values are exposed, the security of the system is compromised [3].*

The reason these random numbers must be deleted immediately is that any entity knowing these values could store them and generate **fake proofs**, thereby bypassing the system’s security.

Given the complexity involved in implementing these protocols from scratch, specialized programming languages called **Domain Specific Languages** (DSLs) have been developed.

The goal is to allow developers who are not cryptography experts to program the statements they wish to prove. It is in this context that **Circom** becomes important—a tool created to facilitate the creation of circuits and their subsequent transformation into constraint systems.

2.2. CIRCOM

Circom is a circuit description language (DSL) whose primary purpose is not computation itself, but defining a system of equations that describes the relationships between input and output **signals** [4].

A signal is a component that acts like a wire within the circuit, carrying values through logic gates and forming part of the equations the programmer intends to verify. The constraints of the equation system generated by Circom describe the behavior of the circuit’s signals.

This perspective is fundamental, as ZKPs cannot interpret code directly; they require computation to be expressed as a structure of algebraic constraints called R1CS, which we will detail in **Section 2.3**.

The Circom compilation process reflects a hybrid paradigm between **declarative and imperative**. Accordingly, the compiler generates two essential files to produce the proof:

- **R1CS File:** Contains the set of all algebraic constraints that define the circuit logic. These constraints are added from a Circom program using the `<==` and `===` symbols.
- **Witness Generator:** An executable (in C++ or WASM format) responsible for calculating the values of all circuit signals, including intermediate values, based on the provided inputs. These assignments are added from a Circom program using the `<==` and `<--` symbols.

Starting from these two files, we can generate the proof using **Snarkjs**. This tool takes the R1CS file and the Witness obtained after compiling a Circom program to generate the associated proof. We will analyze its operation and specific commands in detail in **Section 2.4**.

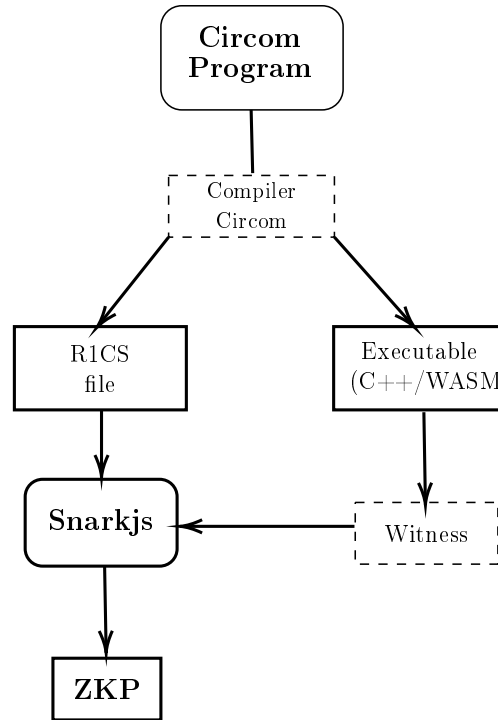


Figura 2.1: Transformation process to ZKP from Circom

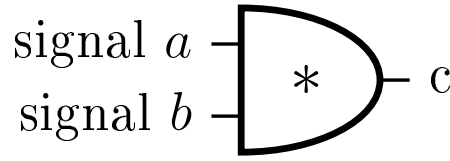
A fundamental characteristic of Circom is that it is a circuit-based language where the circuit structure must be static; this implies that any value determining the circuit's structure (such as array sizes, loop iteration counts, etc.) **must be known at compile time**. This is because the R1CS file contains the algebraic constraints describing the circuit, which cannot depend on the values of the input signals.

As mentioned previously, signals represent the constraints established by the programmer that they wish to verify. Below is an example of a Circom program and its representation as a logic circuit, adding the constraint $a * b = c$.

```

1      template mult(){
2          signal input a;
3          signal input b;
4          signal output c;
5
6          c <== a * b;
7      }
8      main component = mult()
          ;

```



Ejemplo 2.2: Comparative illustration of code vs. circuit

■

The previous example reflects how Circom transforms the equation $c <== a * b$ into an equivalent logic circuit. Thus, we can see that the inputs of the logic component are a and b , thereby forcing c to equal the product of both inputs.

2.3. R1CS

Rank-1 Constraint System (commonly known as **R1CS**) is a mathematical representation system designed to transform complex computational tasks into a set of algebraic constraints. Its relevance lies in the fact that while generating a solution to a problem can be costly, its **verification is extremely efficient and fast**, which is a key requirement for zero-knowledge proofs [5].

For this representation system to be functional, R1CS constraints are defined over a finite field $\mathbb{F}_p$.

Definition 3 (Finite field) *In abstract algebra, a finite field is an algebraic structure containing a finite number of elements. This means that operations applied to these elements are performed using modular arithmetic within a range $[0, p-1]$, where p is a prime number that determines the order of the field [6].*

Circom typically uses very large primes for these finite fields. For instance, one of the primes currently used by Circom corresponds to:

$$p = 52435875175126190479447740508185965837690552500527637822603658699938581184513$$

R1CS imposes specific mathematical constraints. Beyond the fact that equations cannot exceed the second degree (*a necessary but not sufficient condition*), they must be expressible as the product of two linear combinations of variables resulting in a third linear combination:

$$A \cdot B - C = 0$$

Where A , B , and C are linear combinations of the circuit signals. In practical terms, this implies that a single constraint can only contain one multiplication between variables. Operations involving multiple independent products, even if they are of the second degree, must be decomposed into several consecutive constraints using intermediate variables to satisfy this standard.

Following *Example 1.2*, the constraint $c \leq a * b$ is translated into R1CS format as the product of two linear combinations A and B , thereby enforcing that the result of the signals a and b equals c .

Ejemplo 2.3: Multiplication transformation to R1CS format

Given the statement:

$$(a) \times (b) = (c)$$

Where the structure $A \cdot B = C$ is satisfied by assigning:

- $A = a$ (Linear combination 1)
- $B = b$ (Linear combination 2)
- $C = c$ (Result of the constraint)

■

By using the R1CS format, *constraints written in Circom can be verified very efficiently* through Snarkjs, as we will see in **Section 2.4**.

2.4. Snarkjs

Snarkjs is a JavaScript library designed for generating and verifying zero-knowledge proofs from their corresponding R1CS constraints. It can be executed on both mobile devices and servers using *Node.js*, materializing the previously mentioned concept of *1 zk-SNARKs* [4][7].

As previously shown in *Figure 1.1*, Snarkjs requires two files to generate a proof:

- *Witness*, generated by the C++/WASM executable.
- *R1CS File*, containing the circuit's logic constraints.

Once these two files are available and the trusted setup is completed, Snarkjs generates the zero-knowledge proof that allows the Prover to demonstrate the validity of their statement to the Verifier.

2.5. Problem Preamble

Circom is a programming language whose learning curve and application can be complex initially, especially for those lacking experience with constraint-based logic languages. One of the primary reasons for this complexity is the occurrence of what are known as *underconstrained bugs*.

Definition 4 (Underconstrained bug) *An underconstrained bug is an error that occurs when the user fails to include all the necessary equations in the R1CS file to accurately reflect the circuit's intended behavior.*

A Circom program is considered correct when the algebraic equations in the R1CS file exactly represent the circuit's behavior (which is expressed imperatively in the witness generator). The absence of these constraints means that witnesses can be generated that do not reflect the correct behavior of the circuit, yet would still be accepted when verifying the R1CS equations.

The `<--` symbol allows for adding an assignment to the Witness file, but not to the R1CS, as the latter is reserved exclusively for the program's constraints (added using the `<==` and `===` operators).

The functionality of these operators is entirely different. Many people starting out with Circom are initially unclear about the actual difference between these operators, leading to the development of erroneous programs where the generated proof is invalid.

Incorrect use of these operators could lead to problems such as the aforementioned underconstrained bug. If a user utilizes `<--` to add constraints to the equation system instead of `<==` or `===`, those equations will not be added to the R1CS file. Consequently, they will not be part of the equation system describing the circuit, leaving it incomplete.

An example of this type of error can be seen in the following code block, which is intended to check if an input is equal to zero:

```

1     template isZero() {
2         signal input a;
3         signal output b;
4
5         signal inv;
6
7         inv <-- a!= 0 ? 1/a : 0;
8
9         b <== -a*inv * 1;
10    }

```

Código 2.1: Underconstrained bug

This function, while appearing to be correctly implemented, misused the `<--` operator, resulting in an underconstrained bug. By using this operator to calculate the `inv` signal, an assignment is made, but no constraint is added to the equation system.

Because there is no constraint forcing `inv` to be the inverse of `a`, the link between the input `a` and the output `b` is not fully enforced. Therefore, since the assignment `inv <-- a!= 0 ? 1/a : 0` was not added to the equation system, it effectively does not exist for the verifier. This allows a malicious prover to manipulate the value of `inv` to generate fake proofs that the system would accept.

Beyond this specific example, this error recurs frequently in many Circom programs, particularly those written by beginners. Other instances where this error appears include third-party implementations found in public repositories, such as the game "*Battleship*" [8], *Dark Forest* [9], or the *Decoder template in circomlib* [10].

Due to this error, among many others, Circom is a complex language that can be difficult to adapt to, largely because of the various concepts that must be internalized. Therefore, the goal of this work is to provide the user with a much faster, simpler, and more secure way to program in Circom.

By leveraging the flexibility of assignments and the syntax provided by Circom's *declarative-imperative* nature, we can implement new functionality by modifying its compiler. This allows us to translate imperative logic (constraints declared with `<--`, which are normally only added to the *Witness*) into constraints that will form part of the equation system (traditionally only added via `<==` or `==`).

In this way, **the compiler would act as a bridge**, automatically transforming the imperative flow into a system of constraints.

This transformation is achieved by translating Circom's imperative instructions. The process involves analyzing the operators supported by the Circom language and specifying a transformation rule for each one. Through this, the compiler can automatically translate the implementation into a system of constraints.

2.6. Objectives and Work Structure

2.6.1. Objectives

The main objective of this work is to create a simpler and more intuitive way of programming in Circom, making the creation of equation systems and their equivalent logic circuits much more accessible and secure.

Additionally, this work seeks to provide a validation tool. By leveraging the new capabilities of the compiler, it allows the user to compare their own Circom implementation with the one generated through the new functionality, facilitating the verification between their logic and that generated by the compiler.

This final objective also encompasses learning, aiming to reduce the initial complexity of Circom and allowing the user to become familiar with the language in a simpler and more efficient way.

To achieve the proposed objectives, we have carried out the following tasks during the development of this work:

1. Design and specification of the translation rules
2. Implementation of the specified rules within the compiler
3. Verification of the equation systems generated by the rules
4. Application of the new functionality to real-world cases
5. Development of the report to document the above and analyze the results of the work

2.6.2. Work Structure

To achieve the proposed objectives, this document has been structured into the following chapters:

- Chapter 1 (Introduction to Circom): Context is introduced about the technology we are going to deal with and a general overview is offered of what we intend to do with it throughout the work.
- Chapter 2 (Problem definition): The problem will be defined and an example will be presented.
- Chapter 3 (Problem development): Technical specification of the modifications that will be carried out in the language compiler.
- Chapter 4 (Implementation): Detail of the development of the new functionalities in the compiler source code.
- Chapter 5 (Operator verification): Analysis of the verification process to ensure that the new operators generate correct and equivalent constraint systems.

- Chapter 6 (Experiments): Evaluation of the performance and behavior of the modified compiler against manual implementations of simple games and comparison of the new operators with those of `circomlib`.
- Chapter 7 (Conclusions and Future Work): Recapitulation of the obtained results, assessment of the impact of the solution, and proposals for future research.

Definición del problema

3.1. Descripción del problema

La problemática principal que surge durante el proceso de aprendizaje de Circom reside sustancialmente en su diferencia respecto a lenguajes de alto nivel, tanto en objetivo como en restricciones de uso.

Circom, como se ha mencionado previamente, es un lenguaje de bajo nivel, presentando dificultades en su aprendizaje debidas al gran cambio de paradigma respecto a otros lenguajes de programación, más comunes entre los usuarios.

El cambio a un lenguaje basado en circuitos requiere una conversión hacia otra mentalidad, tanto para comprender cuál es el objetivo real de su naturaleza basada en circuitos, así como adaptarse a los recursos ofrecidos por el lenguaje.

Por tanto, se presenta un doble reto:

1. **Asimilar el paradigma de verificación frente al de ejecución**, comprender que el objetivo principal no es realizar un cálculo o cómputo, sino generar un sistema de restricciones con el objetivo de verificar la validez de una demostración.
2. **Adaptarse a la complejidad del diseño algebraico**, procedimiento que implica construir circuitos en los que cada operación tiene que ser expresada a través de restricciones algebraicas, asegurando de esta manera el cumplimiento del formato R1CS.

De esta manera, puede observarse que Circom es un lenguaje que presenta una curva de aprendizaje elevada, de forma que requiere de tiempo y dedicación para desarrollar habilidad y destreza.

Por tanto, para aquellas personas que requieran el uso de este lenguaje, pero se enfrenten a la dificultad del cambio de paradigma se expone una solución cuyo objetivo es facilitar el uso de Circom, explotando su funcionalidad imperativa.

La propuesta radica en la modificación del compilador de Circom para incorporar una nueva funcionalidad que habilite la traducción del código generado por el testigo en un programa Circom a ecuaciones algebraicas en formato R1CS equivalentes.

El código generador del testigo tras la modificación del compilador no sería únicamente funcional para dicha generación, sino que también se permitiría la conversión del código a su equivalente sistema de restricciones R1CS.

Ejemplo 3.1: Implicaciones de la nueva funcionalidad

Compilador sin modificación	Compilador con modificación
<pre>template funcion() { signal input a; signal output b; signal inv; inv <-- a!=0 ? 1/a : 0; b <== -a*inv +1; a*b == 0; }</pre>	<pre>template funcion() autocomplete { signal input a; signal output b; if(a==0){ b <-- 1; } else{ b <-- 0; } }</pre>



El código de la izquierda emplea Circom sin hacer uso de la nueva funcionalidad, donde el programador debe definir explícitamente las restricciones necesarias para reproducir la lógica buscada. Por otro lado, el código de la derecha utiliza la nueva funcionalidad introducida en el compilador del lenguaje, permitiendo el uso de estructuras condicionales así como operadores lógicos. En este caso, el compilador modificado es el encargado de generar las restricciones aritméticas equivalentes a las operaciones imperativas del circuito de partida.

Por esto, ambos bloques de código son equivalentes, ya que el código de la izquierda declara manualmente el sistema de ecuaciones que el código de la derecha va a construir automáticamente.

El primer bloque de código, a pesar de no hacer uso de la nueva funcionalidad, no genera un underconstrained bug aunque utilice el operador `<--`. Aludiendo al *Código 2.1*, puede apreciarse que se diferencian exclusivamente en la restricción `a*b==0`.

Esta restricción permite vincular completamente la entrada `a` con la salida `b`. Su ausencia en el código 2.1 es lo que provoca el mencionado underconstrained bug. Al añadirla se soluciona el error, ya que, a pesar de que `inv` podría adoptar un valor distinto al que se le asignó, la señal `b` queda ahora limitada por el valor de `a` a través de la restricción `a*b==0`, la cual sí se incorpora al sistema de ecuaciones. De este modo, existe una restricción que obliga a la señal `a` a adoptar un único valor, garantizando así un circuito seguro.

Por otro lado, el segundo bloque tiene el mismo comportamiento operacional que el primero, con la única diferencia de que éste hace uso de `autocomplete`. De esta forma se genera un sistema de ecuaciones equivalente al del primer bloque.

El segundo bloque de código emplea una sentencia `if-else` para determinar si la entrada es igual a 0 o no.

Puede apreciarse que la lógica necesaria para implementar una función tan básica sin las modificaciones se convierte en una tarea más tediosa, mientras que con las modificaciones esta lógica se convierte en un simple bloque condicional que cualquier persona familiarizada con la programación sabría hacer.

De esta forma, Circom sería un lenguaje de programación mucho mas accesible para los usuarios, al facilitar su uso incorporando esta nueva funcionalidad.

Para ejemplificar de manera práctica las restricciones mencionadas y la dificultad que supone definir manualmente limitaciones más allá del *Ejemplo 3.1*, se presenta a continuación un ejemplo que contrasta el flujo actual de Circom frente a la solución propuesta.

3.2. Ejemplo: Piedra, Papel y Tijera

El programa que utilizaremos como ejemplo es el clásico juego de *piedra, papel y tijera*. Éste consiste en 3 reglas principales.

1. Piedra gana a Tijera, pero pierde contra Papel
2. Papel gana a Piedra, pero pierde contra Tijera.
3. Tijera gana contra Papel, pero pierde contra Piedra.

Aunque las normas de este juego son extremadamente sencillas, traducirlas al lenguaje de Circom no es una tarea sencilla, ya que requiere convertir una lógica simple en un sistema de restricciones mucho más complejo.

Con el fin de aclarar esta transición entre la lógica del juego y su implementación técnica, se ha desarrollado un repositorio específico que alberga el código fuente del circuito, implementado tanto con la nueva funcionalidad del compilador como sin ella [11]. A continuación, se muestra un fragmento de este programa Circom equivalente a la lógica del juego:

```

1      template piedra_papel_tijera 30      eq1 <== (J1_1.out)*(J2_1.
      () {                                out);
2      signal input jugador1;           31      eq2 <== (J1_2.out)*(J2_2.
3      signal input jugador2;           out);
4      signal output ganador;           32      eq <== eq0 + eq1 + eq2;
5                                          33
6      signal eq; signal inv;           34      // Inversion de empate
7      signal out; signal j1;           35      inv <== (1 - eq);
8      signal j2;                       36
9                                          37      // Ganadoras Jugador 1
10     // Jugador 1                     38     signal j1_0; signal j1_1;
11     component J1_0 = igualdad         39     signal j1_2;
      (0);                               40     j1_0 <== (J1_0.out)*(J2_2.
12     J1_0.in <== jugador1;             out);
13     component J1_1 = igualdad         41     j1_1 <== (J1_1.out)*(J2_0.
      (1);                               out);
14     J1_1.in <== jugador1;             42     j1_2 <== (J1_2.out)*(J2_1.
15     component J1_2 = igualdad         out);
      (2);                               43     j1 <== j1_0+j1_1+j1_2;
16     J1_2.in <== jugador1;             44
17                                          45     // Ganadoras Jugador 2
18     // Jugador 2                     46     signal j2_0; signal j2_1;
19     component J2_0 = igualdad         47     signal j2_2;
      (0);                               48     j2_0 <== (J1_0.out)*(J2_1.
20     J2_0.in <== jugador2;             out);
21     component J2_1 = igualdad         49     j2_1 <== (J1_1.out)*(J2_2.
      (1);                               out);
22     J2_1.in <== jugador2;             50     j2_2 <== (J1_2.out)*(J2_0.
23     component J2_2 = igualdad         out);
      (2);                               51     j2 <== j2_0+j2_1+j2_2;
24     J2_2.in <== jugador2;             52
25                                          53     // Calculo final
26     // Igualdad (empate)              54     out <== j2;
27     signal eq0; signal eq1;           55     ganador <== ((1-inv)*2)+(
28     signal eq2;                       out*inv);
29     eq0 <== (J1_0.out)*(J2_0.         }
      out);

```

Figura 3.1: Implementación de Piedra, papel y tijera

Para simular la lógica del juego partimos de 2 señales de entrada (L1-4), que representan los dos jugadores (jugador1 y jugador2), y una señal de salida (ganador).

3.2.1. Señales de los jugadores

Las señales jugador1 y jugador2 representan cuál ha sido el elemento escogido por cada jugador. Estas señales están limitadas a 3 valores, cuyo significado es el siguiente:

1. Piedra: 0
2. Papel: 1
3. Tijera: 2

Por ejemplo, si el jugador1 elige piedra y el jugador2 elige tijera, los valores de estas señales serán jugador1 = 0 y jugador2 = 2.

Como se ha mencionado en el capítulo anterior, el valor de las señales debe conocerse en tiempo de compilación para poder construir correctamente el circuito. Para determinar qué elemento ha elegido cada jugador, es necesario instanciar componentes, que en Circom actúan como llamadas a otros *templates* (plantillas) definidos previamente. En este caso, se declaran tres componentes por cada jugador para identificar de forma única su elección: J1_0, J1_1 y J1_2 para el primero, y J2_0, J2_1 y J2_2 para el segundo.

De esta manera, seremos capaces de determinar cuál fue el elemento elegido por cada jugador y, basándonos en las reglas del juego, podremos establecer si se trata de una victoria del jugador 1, una victoria del jugador 2 o un empate.

Un ejemplo concreto de esta lógica es la determinación del empate (L26-32). Para identificar esta situación, se declaran tres señales intermedias (`eq0`, `eq1` y `eq2`) que actúan como indicadores de igualdad para cada una de las opciones:

- `eq0` (L29): Multiplica las salidas de J1_0 y J2_0. Solo será 1 si ambos jugadores eligieron piedra (0).
- `eq1` (L30): Multiplica las salidas de J1_1 y J2_1. Solo será 1 si ambos eligieron papel (1).
- `eq2` (L31): Multiplica las salidas de J1_2 y J2_2. Solo será 1 si ambos eligieron tijera (2).

3.2.2. Señal del ganador

Por otro lado, la señal `ganador` representa el resultado final del juego, teniendo la posibilidad de tomar tres valores distintos dependiendo del desempeño del juego:

1. Victoria del Jugador 1: 0
2. Victoria del Jugador 2: 1
3. Empate: 2. Este valor se genera mediante la interacción de la señal `inv` (L35) y la expresión de la L55. La variable `inv` representa la inversión del empate (`1 - eq`); por tanto, si hay empate, `inv` vale 0. En la expresión final:

$$\text{ganador} \leftarrow ((1 - \text{inv}) \cdot 2) + (\text{out} \cdot \text{inv})$$

Si se produce el empate, el término `(out * inv)` se anula al multiplicarse por cero, mientras que `(1 - inv) * 2` se convierte en `(1 - 0) * 2 = 2`, garantizando que el resultado sea el convenio establecido para el empate.

Una vez definida la función de cada señal y el significado de cada componente, la estrategia para reproducir la lógica del juego resulta directa mediante el uso de selectores aritméticos.

En esta estrategia, las señales `j1` (L43) y `j2` (L51) actúan como acumuladores de victoria. Por ejemplo, la variable `j2` es el resultado de sumar los productos de las

señales de igualdad que representan los casos donde el Jugador 2 gana (Piedra vs Papel, etc.). Al ser casos excluyentes, `j2` solo puede ser 1 (si el Jugador 2 gana) o 0 (si pierde o empata), restringiendo a estas señales a un valor binario debido a la naturaleza disyuntiva de las reglas del juego.

La variable `out` (L54) captura este estado de victoria del Jugador 2. De este modo, la expresión de la señal `ganador` utiliza `out` e `inv` como llaves de paso:

- Si no hay empate (`inv=1`), la salida será directamente el valor de `out=2`.
- Si el Jugador 2 ganó, `out=1` y la salida es 1.
- Si el Jugador 2 no ganó (y sabemos que no hay empate), por eliminación el ganador es el Jugador 1, `out=0` y la salida es 0.

Mediante la combinación de señales de igualdad y operaciones de suma y producto, el circuito calcula el resultado de manera sistemática. Así, la determinación del ganador se consigue por medio de la interacción entre las señales que están activas, es decir, las señales cuyo valor es 1

Tras la introducción del código equivalente a la lógica del juego, puede apreciarse a simple vista la complejidad de describir flujos lógicos usando Circom, requiriendo tanto de maestría en su sintaxis y recursos, como de destreza para reflejar fielmente la lógica del juego.

Sin embargo, la incorporación de nuestra modificación sobre el compilador simplificará este proceso, permitiendo una programación más intuitiva sin privar al lenguaje de las restricciones algebraicas necesarias para generar la prueba de conocimiento cero.

A continuación, vamos a ver la comparativa entre ambos códigos más detalladamente, comenzando por el empate.

3.2.3. Lógica del empate

Ejemplo 3.2: Traducción de la lógica de empate

Circom Estándar	Modificación (autocomplete)
<pre>signal eq0, eq1, eq2; eq0 <== (J1_0.out) * (J2_0.out); eq1 <== (J1_1.out) * (J2_1.out); eq2 <== (J1_2.out) * (J2_2.out); eq <== eq0 + eq1 + eq2; inv <== (1 - eq);</pre>	<pre>if(jugador1 == jugador2){ ganador <-- 2; }</pre>



Como puede observarse, incluso en una verificación de igualdad como es el empate, el usuario debe manejar señales auxiliares e inversas (`inv`).

Para implementar la representación de la lógica del empate sin recurrir al uso de la nueva funcionalidad, es necesario implementar un flujo utilizando operaciones de suma y multiplicación. Como hemos mencionado, en este juego, solo pueden ocurrir tres circunstancias de empate.

Por lo tanto, es necesario crear un componente para cada circunstancia que represente ese empate. Esta lógica se consigue multiplicando el valor de las señales `jugador1` y `jugador2` en la correspondiente situación de empate.

Únicamente se activará una señal eq_n cuando la salida del componente correspondiente sea 1 para ambos jugadores; de lo contrario, si solo un jugador la tiene activa, el resultado de la multiplicación será 0

De la misma forma, sería imposible que varias señales estuviesen activas, ya que esta situación de empate solo puede darse para una señal eq_n .

Por ejemplo, en la situación de empate piedra—piedra, el valor de `eq0` sería 1. En esta situación, `eq` debe ser exclusivamente 1; si `eq0` es 1 (debido a que `J1_0.out=1` y `J2_0.out=1`), se imposibilita que `eq1` o `eq2` sean 1, ya que un jugador no puede haber seleccionado dos elementos simultáneamente.

Esta necesidad de asegurar la exclusividad entre las distintas señales mediante restricciones aritméticas añade una carga de complejidad notable, la cual se elimina por completo con la nueva funcionalidad.

Sin embargo, toda esta complejidad lógica se reduce considerablemente haciendo uso de la nueva modificación, ya que se minimiza esta dificultad. Al permitir el

uso de operadores como `==` y expresiones condicionales, únicamente es necesaria una condición que compruebe si `jugador1 == jugador2` y, en tal caso, asignar qué valor tendrá la señal `ganador`.

Esta complejidad aumenta exponencialmente al definir las condiciones de victoria de los jugadores en el **Ejemplo 3.3**.

3.2.4. Lógica de los jugadores

Ejemplo 3.3: Traducción de la lógica del jugador 1

Circom Estándar	Modificación (autocomplete)
<pre> signal j1_0; signal j1_1; signal j1_2; j1_0 <== (J1_0.out) * (J2_2.out); j1_1 <== (J1_1.out) * (J2_0.out); j1_2 <== (J1_2.out) * (J2_1.out); j1 <== j1_0 + j1_1 + j1_2; </pre>	<pre> else if(jugador1 == 0 && jugador2 == 2){ ganador <-- 0; } else if(jugador1 == 1 && jugador2 == 0){ ganador <-- 0; } else if(jugador1 == 2 && jugador2 == 1){ ganador <-- 0; } </pre>



Ambos bloques de código son equivalentes. El bloque de código de la izquierda íntegramente escrito con el operador `<==` para incorporar todas las restricciones al sistema de ecuaciones. El código de la derecha, aunque no utiliza el operador `<==`, puede usar el operador `<--` sin inconvenientes debido a las modificaciones que hemos añadido sobre el compilador. De este modo, las restricciones se añaden también al archivo R1CS y no solo al testigo.

Por lo tanto, puede apreciarse que el esfuerzo lógico para describir la lógica del juego utilizando el operador `<==` es mucho más complejo que aprovechando la nueva funcionalidad añadida. Su uso evita que el programador tenga que transformar manualmente cada bifurcación del algoritmo en expresiones algebraicas, delegando ese trabajo al compilador modificado.

Tras analizar ambos bloques de código, es evidente la simplicidad y legibilidad que la nueva funcionalidad incorpora al lenguaje. Mientras que el original exige instanciar múltiples componentes y realizar productos de señales para comprobar cada estado, los nuevos condicionales introducidos por la nueva funcionalidad permiten

una traducción directa que ofrece al programador una mayor libertad a la hora de construir circuitos.

Si bien este beneficio es ya notable en un ejemplo sencillo como este, su verdadero potencial reside en su aplicación a problemas más complejos.

Al trasladar esta lógica a sistemas de mayor envergadura, se mejora la mantenibilidad y la accesibilidad en el diseño de circuitos a la vez que se reducen las posibles situaciones de error.

Desarrollo del problema

4.1. Síntesis de restricciones para la traducción de expresiones

A lo largo del documento estudiaremos cómo traducir instrucciones y expresiones generadas por un lenguaje imperativo simple en un conjunto de ecuaciones en formato R1CS equivalente. Para ello definimos las siguientes funciones:

Definición 5 (Traducción de expresiones) *Sea $\mathcal{C}$ un conjunto de restricciones en formato R1CS. Definimos el conjunto de expresiones $Expr$ de forma inductiva mediante la siguiente sintaxis:*

$$e := c \mid x \mid e_1 \oplus_{bin} e_2 \mid \ominus_{un} e$$

donde c es una constante, x es una variable, y los operadores se dividen en:

- $\oplus_{bin} \in \{+, -, \times, /, \%, \&\&, ||, \&, |, <, >, \leq, \geq, ==, \neq, \ll, \gg\}$ representa los operadores binarios.
- $\ominus_{un} \in \{!, -\}$ representa los operadores unarios.

Definimos la función de traducción $TC : Expr \rightarrow \mathcal{C} \times Expr$ tal que para cada expresión $e \in Expr$:

$$TC(e) = (C, e')$$

donde $C \in \mathcal{C}$ es el conjunto de restricciones acumuladas y e' es la expresión que representa el resultado de la expresión original.

Esta definición es necesaria para la especificación de las reglas aplicadas sobre cada expresión, teniendo como intuición que si las constraints C de cada expresión e son verificadas exitosamente, entonces e y e' siempre tendrán el mismo valor.

Vamos a definir la función TC de forma inductiva, comenzando por los casos base (números y variables), para luego pasar a definir los casos compuestos respectivos a los operadores.

Casos base

- Si la expresión es un número n :

$$TC(n) = TC(\emptyset, n)$$

- Si la expresión es una variable v :

$$TC(v) = TC(\emptyset, v)$$

4.1.1. Casos inductivos: Reglas de transformación de expresiones

Esta sección recoge la documentación sobre la implementación de las reglas de transformación pertinentes para materializar la funcionalidad descrita a la práctica.

Las reglas de transformación especificadas abarcan los operadores lógicos, de comparación, aritméticos y desplazamiento.

4.1.1.1. Operadores Aritméticos

Traducción de operadores

La formalización de los operadores aritméticos se establece mediante un sistema de reglas de deducción que definen la traducción de cada operación al lenguaje de restricciones del circuito. En la siguiente tabla se detallan estas transformaciones, especificando cómo se generan las señales y restricciones necesarias:

Operación	Regla de Transformación
$e = a + b$	$\frac{TC(a) = (C_1, a') \quad TC(b) = (C_2, b')}{TC(a + b) = (C_1 \wedge C_2, a' + b')}$
$e = a - b$	$\frac{TC(a) = (C_1, a') \quad TC(b) = (C_2, b')}{TC(a - b) = (C_1 \wedge C_2, a' - b')}$
$e = a * b$	$\frac{TC(a) = (C_1, a') \quad TC(b) = (C_2, b')}{TC(a * b) = (C_1 \wedge C_2 \wedge \{v_{aux} = a' * b'\}, v_{aux})}$

Operación	Regla de Transformación
$e = a / b$	$TC(a) = (C_1, a') \quad TC(b) = (C_2, b') \quad q', r' \text{ son señales intermedias}$ $TC(a/b) = (C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} b' \neq 0, \\ 0 \leq r' < b' , \\ v_{aux} = b' \cdot q', \\ a' === v_{aux} + r' \end{array} \right\}, q')$
$e = a \% b$	$TC(a) = (C_1, a') \quad TC(b) = (C_2, b') \quad q', r' \text{ son señales intermedias}$ $TC(a \% b) = (C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} b' \neq 0, \\ 0 \leq r' < b' , \\ v_{aux} = b' \cdot q', \\ a' === v_{aux} + r' \end{array} \right\}, r')$

Los operadores de **suma** y **resta** son los más sencillos de implementar debido a su naturaleza, ya que ninguna de estas operaciones corre el riesgo de generar declaraciones que incumplan el formato R1CS. Por tanto, no se requieren señales intermedias para asegurar el cumplimiento de los estándares algebraicos.

Esto no aplica a la **multiplicación**. El motivo de esta diferencia con respecto a la **suma** y **resta** se debe a que existe una incertidumbre respecto al grado de la declaración. Al tratarse de un producto, no se puede garantizar que sus operandos no provengan de otra operación cuyo grado sea cuadrático.

En consecuencia, cualquier operador que corra el riesgo de exceder el segundo grado en una declaración, y por tanto de incumplir el formato establecido por R1CS, debe utilizar señales intermedias para así cumplir dicho formato.

$$mul = a \cdot b$$

$$signal - mul = 0$$

El uso de la señal intermedia **signal** permite asegurar que una multiplicación siempre sea lineal, cumpliendo así el formato R1CS.

Por otro lado, operadores como la **división** y el **módulo** plantean una implementación más compleja, ya que para ambos operadores es necesario el desarrollo y cumplimiento del *teorema de la división*, definido en la siguiente sección.

División

La especificación de la división es posible gracias al teorema de la división, el cual dice lo siguiente:

Definición 6 (Teorema de la división) *Dados dos enteros a (dividendo) y b (divisor), con $b \neq 0$, existen únicamente dos enteros q y r tales que:*

$$\begin{aligned}a &= b \cdot q + r \\ 0 &\leq r < |b|\end{aligned}$$

Donde:

- q representa el **cociente** y r representa el **resto**.

Conociendo este teorema [12], en las siguientes secciones se especificará la aplicación de este teorema para conseguir la traducción a las restricciones algebraicas deseadas para los operadores división y módulo.

En Circom, trabajar sobre un cuerpo finito $\mathbb{F}_p$ implica que el valor de todas las señales debe ser un elemento de este conjunto, restringiendo de esta manera los valores al campo de los enteros módulo p . Por este motivo, la división decimal nunca existirá en este contexto. En su lugar, la división se calcula utilizando el inverso multiplicativo del divisor.

Ejemplo 1 *Si aplicamos esto a un caso concreto, supongamos que trabajamos en el cuerpo finito $\mathbb{F}_5$ y queremos calcular la división $3 / -2$.*

En este contexto, primero debemos transformar el número negativo -2 a su equivalente positivo dentro del cuerpo, lo cual se hace sumándole el módulo ($5 - 2 = 3$), por lo que $-2 \equiv 3 \pmod{5}$. A partir de ahí, buscamos el inverso multiplicativo de 3, que es 2 (ya que $(3 \times 2) \pmod{5} = 1$).

Por lo tanto, la operación se resuelve como una multiplicación por dicho inverso:

$$3 / -2 = 3/3 = 3 \times 3^{-1} = 3 \times 2 = 6 \pmod{5} = 1$$

De esta manera, el resultado de la división sigue siendo un número entero dentro del cuerpo (en este caso, 1).

Por tanto, en esta situación es el cociente q el valor que buscamos para esta operación. Esto no significa que el resto r sea despreciado, ya que es fundamental para satisfacer las restricciones $a = b \cdot q + r$ y $0 \leq r < |b|$. Únicamente con el cumplimiento de estas restricciones se garantiza que el valor de q sea único y correcto según el teorema.

De forma que el resto r tomará el valor correspondiente para satisfacer la expresión $a = b \cdot q + r$, al igual que el cociente q , cumpliendo así el teorema y obteniendo el valor q buscado.

Para simular el comportamiento de este operador, el sistema de ecuaciones no solo define $a = b \cdot q + r$, sino que impone las condiciones $b \neq 0$ y $0 \leq r < |b|$ para garantizar la satisfacibilidad del sistema de ecuaciones.

De esta manera, para conseguir que el cociente q obtenga el valor esperado, se realizan las siguientes transformaciones sobre la expresión del teorema:

$$a = b \cdot q + r \quad (4.1)$$

$$a - (b \cdot q + r) = 0 \quad (4.2)$$

Así, al obligar a que la expresión sea igual a cero, garantizamos que el cociente q y el resto r tomen los valores exactos que satisfacen el teorema dentro del sistema.

Ejemplo 2 *Para ilustrar el funcionamiento de la división, supongamos el caso de $a = 15$ y $b = 3$. La lógica implementada para este operador debe seguir los siguientes pasos:*

1. Comprobación entera positiva

Primeramente, se verifica que a y b sean números enteros positivos mediante la función Num2Bits(). Esta comprobación es fundamental para evitar errores de desbordamiento en el campo finito. En este caso, $15 > 0$ y $3 > 0$, cumpliendo la condición.

2. Validación del divisor

Se verifica que $b \neq 0$ para asegurar que la operación esté definida. Al ser $b = 3$, el circuito permite continuar con la ejecución sin lanzar una excepción de división por cero.

3. Restricción del resto

Se impone la condición $r < b$. Es importante observar que no es necesario verificar explícitamente $0 \leq r$, ya que en Circom las señales se asumen positivas por defecto dentro del rango del módulo. Para $q = 5$ y $r = 0$, comprobamos que $0 < 3$.

4. Ecuación de la división

Finalmente, se establece la restricción que define el comportamiento del circuito:

$$a = b \cdot q + r$$

Las señales q (cociente) e r (resto) son forzadas a tomar los únicos valores que satisfacen el sistema. En este ejemplo, las restricciones solo admiten la solución $q = 5$ y $r = 0$.

■

Una vez conocida cuál es la lógica necesaria para simular el comportamiento de la división y expuesto un ejemplo de como se aplicarían las restricciones para unos valores de entrada concretos, se presente un código escrito en Circom base para ilustrar y clarificar como sería el flujo de esta lógica sin hacer uso de la nueva funcionalidad.

Ejemplo 4.1: Pseudocódigo división

```

1      template IntegerModule() {
2          signal input a; // Dividendo
3          signal input b; // Divisor
4          signal output c; // Cociente q
5
6          signal r;
7          signal q;
8
9          // Comprobamos que b!= 0
10         component eq = IsZero();
11
12         eq.in <== b;
13         eq.out === 0;
14
15         // Comprobamos que 0<= r < |b|
16         // 0 <= r no se comprueba porque es implicito
17
18         component lt = LessThan();
19         lt[0].in<==r;
20         lt[1].in<==b;
21         lt.out === 1;
22
23         // Forzamos a que se cumpla a - (b*q+r) = 0
24         a === b * q + r;
25         c <== q;
26     }
27
28     component main = IntegerDivision();

```

Las funciones `IsZero()` y `LessThan()` se utilizan para comprobar si un valor es cero y para comparar si un valor es menor que otro, respectivamente. Ambos componentes pertenecen a `circomlib` [13], una de las librerías de componentes reutilizables más extensas y utilizadas en la comunidad.

Para conseguir esta lógica en Circom sin hacer uso de la nueva funcionalidad puede apreciarse que requiere de un diseño aritmético complejo, ya que habría que establecer cada una de las condiciones a mano para simular este comportamiento.

Módulo

El comportamiento del operador módulo se obtiene de manera equivalente a la división, con la diferencia de que el valor buscado ya no es el cociente q , sino el resto r . Por tanto, el sistema de ecuaciones es exactamente el mismo, pero devolviendo como valor final dicho resto.

Ejemplo 4.2: Pseudocódigo módulo

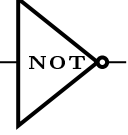
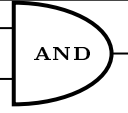
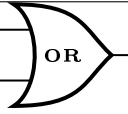
```
1      template IntegerModulo) {
2          // Codigo previo, igual que el de la division
3          ...
4          // Forzamos a que se cumpla a - (b*q+r) = 0
5          a == b * q + r;
6          c <= r; //Observar que ya no es q
7      }
8
9      component main = IntegerDivision();
```

4.1.1.2. Operadores Lógicos

Los operadores lógicos se implementan por medio de la reproducción de las puertas lógicas AND, OR y NOT.

Las puertas lógicas tienen un equivalencia aritmética que simula el comportamiento de las mismas, en el casos de las puertas lógicas NOT, AND y OR su equivalencia es la siguiente:

Cuadro 4.2: Relación entre simbología lógica identificada y su traducción aritmética.

Símbolo Lógico	Equivalencia Aritmética
	$1 - a$
	$a \cdot b$
	$a + b - (a \cdot b)$

De esta forma, podemos explotar esta equivalencia aritmética para la traducción de

estos operadores, ya que con esta equivalencia podemos declarar las restricciones necesarias su implementación.

De la misma forma podemos aplicar esta lógica a los operadores binarios. Los operadores **NOT**, **AND** y **OR** se aplican a un valor o par de valores mientras que los operadores **AND** y **OR** **bit a bit** se aplican sobre cada par de bits de dos números binarios.

Por lo tanto, si partimos de dos números binarios de tamaño n , estos operadores se aplicarán entre cada par de bits. Por ejemplo, el operador **AND** **binario** se aplicaría de la siguiente forma:

$$\text{Siendo } i \in [0, n) \\ b1[i] \cdot b2[i]$$

De forma análoga se aplicaría el operador **OR** utilizando la equivalencia aritmética de dicha puerta lógica para cada par de bits.

Traducción de operadores

Operación	Regla de Transformación
not e_1	$\frac{TC(e_1) = (C_1, e'_1)}{TC(\text{not } e_1) = (C_1 \wedge \{e'_1 \cdot (e'_1 - 1) == 0\}, 1 - e'_1)}$
e_1 and e_2	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \text{ and } e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} e'_1 \cdot (e'_1 - 1) == 0, \\ e'_2 \cdot (e'_2 - 1) == 0, \\ v_{aux} = e'_1 \cdot e'_2 \end{array} \right\}, v_{aux} \right)}$
e_1 or e_2	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \text{ or } e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} e'_1 \cdot (e'_1 - 1) == 0, \\ e'_2 \cdot (e'_2 - 1) == 0, \\ v_{aux1} = e'_1 \cdot e'_2, \\ v_{aux} == e'_1 + e'_2 - v_{aux1}, \end{array} \right\}, v_{aux} \right)}$
e_1 & e_2	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \& e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} \text{BinaryAnd}(n).in[0] <== e'_1 \\ \text{BinaryAnd}(n).in[1] <== e'_2 \\ \text{BinaryAnd}(n).out = v_{aux} \end{array} \right\}, v_{aux} \right)}$
e_1 e_2	$\frac{TC(e_1) = (C_1, e'_{1,bin}) \quad TC(e_2) = (C_2, e'_{2,bin})}{TC(e_1 e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} \text{BinaryOr}(n).in[0] <== e'_1 \\ \text{BinaryOr}(n).in[1] <== e'_2 \\ \text{BinaryOr}(n).out = v_{aux} \end{array} \right\}, v_{aux} \right)}$

La función $\text{BinaryAnd}(n)$ recibe dos números decimales (e'_1 y e'_2), de forma que:

1. e'_1 es transformado a binario: $e'_{1_{bin}}$
2. e'_2 es transformado a binario: $e'_{2_{bin}}$
3. Una vez tenemos los números binarios transformados, podemos aplicar la operación **AND** entre cada par de bits: $result[i] = e'[i]_{1_{bin}} \cdot e'[i]_{2_{bin}}$, donde $i \in [0, n)$
4. Una vez aplicada la operación **AND**, **result** es transformado a decimal con la función $Bits2Num()$, la cuál de forma análoga a $Num2Bits()$ transforma el número binario a decimal.

La función $BinaryOr(n)$ recibe dos números decimales (e'_1 y e'_2), de forma que:

1. e'_1 es transformado a binario: $e'_{1_{bin}}$
2. e'_2 es transformado a binario: $e'_{2_{bin}}$
3. Una vez tenemos los números binarios transformados, podemos aplicar la operación **OR** entre cada par de bits: $result[i] = e'[i]_{1_{bin}} + e'[i]_{2_{bin}} - e'[i]_{1_{bin}} \cdot e'[i]_{2_{bin}}$, donde $i \in [0, n)$
4. Una vez aplicada la operación **OR**, **result** es transformado a decimal con la función $Bits2Num()$, la cuál de forma análoga a $Num2Bits()$ transforma el número binario a decimal.

4.1.1.3. Operadores de Comparación

Tras establecer la base de los operadores aritméticos y lógicos, es fundamental definir la lógica correspondiente a los operadores de comparación. En la siguiente tabla se formalizan las reglas de traducción para los operadores relacionales y de igualdad:

Operación	Regla de Transformación
$e = e_1 == e_2$	$TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)$ $TC(e_1 == e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} EQI().in[0] = e'_1, \\ EQI().in[1] = e'_2, \\ EQI().out = v_{aux} \end{array} \right\}, v_{aux} \right)$
$e = e_1 \neq e_2$	$TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)$ $TC(e_1 \neq e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} EQI().in[0] = e'_1, \\ EQI().in[1] = e'_2, \\ EQI().out = v_{aux} \end{array} \right\}, 1 - v_{aux} \right)$
$e = e_1 <_{(n)} e_2$	$TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)$ $TC(e_1 <_{(n)} e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} SLTEq(n).in1 = e'_2, \\ SLTEq(n).in2 = e'_1, \\ SLTEq(n).out = v_{aux} \end{array} \right\}, 1 - v_{aux} \right)$

Continúa en la siguiente página...

Operación	Regla de Transformación (cont.)
$e = e_1 \leq_{(n)} e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \leq_{(n)} e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} SLTEq(n).in1 = e'_1, \\ SLTEq(n).in2 = e'_2, \\ SLTEq(n).out = v_{aux} \end{array} \right\}, v_{aux} \right)}$
$e = e_1 >_{(n)} e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 >_{(n)} e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} SLTEq(n).in1 = e'_1, \\ SLTEq(n).in2 = e'_2, \\ SLTEq(n).out = v_{aux} \end{array} \right\}, 1 - v_{aux} \right)}$
$e = e_1 \geq_{(n)} e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \geq_{(n)} e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} SLTEq(n).in1 = e'_2, \\ SLTEq(n).in2 = e'_1, \\ SLTEq(n).out = v_{aux} \end{array} \right\}, v_{aux} \right)}$

Los operadores $==$ y $!=$ no dependen del valor de n , por lo que el operador $op_{(n)}$ no se aplica a ellos. En cambio, para el resto de los operadores sí aplica. De este modo, si el programador no lo utiliza, se asumirá por defecto un tamaño de $n = 253$.

Las siglas *SLTEq* denotan *SecureLessThanEq()*, función que simula el comportamiento explicado en la **Operadores** $<, \leq, > \ y \geq$ para dos entradas.

Por otro lado, EQI denota las siglas *Equality Indicator*, haciendo referencia a una función encargada de imitar al comportamiento explicado en el apartado de **Operadores** $== \ y !=$ para dos entradas.

Observación: Es importante darse cuenta que podemos hacer uso de la misma función para los operadores $<, \leq, > \ y \geq$ explotando su carácter reflexivo. Para ello, notesé que las llamadas de los operadores $< \ y \geq$ se invierten.

Tamaño de los números comparados

El tamaño de los números comparados es un factor importante, ya que dicho tamaño puede ser por un lado conocido e igual, mientras que por otro lado puede ser desconocido o de diferentes tamaños.

Un tamaño es desconocido cuando una señal no tiene una restricción de bits predefinida, existiendo únicamente como un valor dentro del rango del cuerpo finito $\mathbb{F}_p$ ($[0, p-1]$).

Para resolver este problema, vamos a definir un complemento opcional por cada operador de comparación existente.

Dicho complemento consiste en restringir el tamaño de los números comparados a un tamaño especificado por el usuario. Si el usuario no hace uso de este complemento opcional para la especificación del tamaño de los números a comparar, se utilizará el tamaño máximo por defecto, correspondiente a 253.

Definición 7 (Complemento opcional de tamaño) *Vamos a definir el operador:*

Operador *op*

- *op*: Comparaciones de cuyos números se conoce el tamaño y además tienen el mismo tamaño.
- La ausencia de dicho componente se traduce a la asunción del 253 como el tamaño fijo de ambos números comparados, el mayor tamaño posible.

Operador *op__n*

- *op__n*: Comparaciones de cuyos números no se conoce el tamaño y/o no tiene el mismo tamaño. De esta manera, *n* indica el tamaño fijado para ambos números comparados.

Este complemento aplica para todos los operadores de comparación, siguiendo la misma estructura que la del *Ejemplo 7*.

Un caso de uso se presenta a continuación para clarificar la naturaleza de dicho complemento, así como su aplicación.

```
1      template comparacion(n) autocomplete{
2          signal input a;
3          signal output b;
4
5          if(a >_(2) b){
6              b <-- 2;
7          }
8          else if(a ==_(2) b){
9              b <-- 1;
10         }
11         else{
12             b<--0;
13         }
14     }
```

Ejemplo 4.3: Caso de uso del complemento opcional de tamaño



Siguiendo el ejemplo anterior, la introducción del operador *op__n* permite limitar el tamaño en bits de los operadores > y ==, restringiendo así el tamaño de las señales *a* y *b* a *n*=2.

Este operador resulta de gran utilidad cuando el programador conoce previamente el tamaño de las señales. Si no estuviese disponible, el sistema utilizaría por defecto el tamaño máximo de 253 bits.

Aunque el usuario conociese que las señales con las que va a trabajar no exceden un tamaño de 2, se generarían 251 señales innecesarias para este circuito, afectando considerablemente a su eficiencia.

Por ello, si el usuario es conocedor del tamaño, es una herramienta que reduce considerablemente el número de restricciones generadas.

Tratamiento de bits

Debido a la forma en la que Circom gestiona los valores numéricos, los números negativos requieren un tratamiento especial (véase el **Anexo A**).

Para solventar este problema, recurrimos a la transformación de los números a comparar de sistema decimal a binario. Esta conversión es estrictamente necesaria para asegurar que los números correspondientes no exceden el rango especificado.

La función *Num2Bits(n)* consigue cumplir esta precondition ya que su funcionamiento radica en la transformación numérica de decimal a binario, reconstruyendo el número decimal entrante durante la misma.

Num2Bits(n):

```

1      template Num2Bits(n){
2          signal input in;
3          signal output out[n];
4          var lc1=0; // Reconstruye el numero decimal a
                    convertir para la constraint
5
6          var e2=1; // Evitar la potencia
7          for (var i = 0; i<n; i++) {
8              out[i] <-- (in >> i) & 1; // Contruimos el numero
                    en binario
9              out[i] * (out[i] -1 ) == 0;
10             lc1 += out[i] * e2;
11             e2 = e2+e2;
12         }
13
14         lc1 == in; // Constraint permite distinguir entre
                    num.positivo o negativo
15     }

```

Código 4.1: Función Num2Bits(n)

Este bloque de código corresponde a la plantilla *Num2Bits()* de la librería *circomlib* [14]. Esta función ha sido el modelo de referencia para implementar la conversión decimal–binaria dentro de la nueva funcionalidad del compilador desarrollado.

De esta forma, al terminar la transformación y antes de salir de la función, se realiza una comprobación para verificar si el número decimal entrante y el número decimal reconstruido son iguales.

Debido al excesivo tamaño del número negativo en el campo finito, esta restricción de igualdad nunca se cumplirá. Como el número de bits fijado (*n*) es mucho menor

que el tamaño real del número negativo, la reconstrucción solo recupera una fracción del valor, resultando en un número decimal distinto al original.

Por tanto, no habrá un bucle que reconstruya un número decimal negativo ya que nunca se realizan las iteraciones necesarias, forzando de esta forma a que solo los números positivos cumplan dicha condición.

Intuición del funcionamiento de $Num2Bits(n)$ con un número negativo entrante:

1. El número negativo se interpreta como el módulo del campo.
2. Éste se interpretará como un número superior a $2^n - 1$, saliéndose fuera del rango $[0, 2^n - 1]$, por tanto la restricción $lc1 === in$ de la función nunca se cumplirá en este caso y la prueba no se generará.

En conclusión, $Num2Bits()$ garantiza el uso de número positivos dentro del rango mencionado de manera implícita.

Un ejemplo para ilustrar el comportamiento de la función en caso de número negativo es el siguiente:

Ejemplo 4.4: Comportamiento Num2Bits(10)

1. Partimos de un número negativo, por ejemplo -3. Este valor, al trabajar sobre un cuerpo finito se codifica como $p-3$.
2. La función ejecuta un bucle de 10 iteraciones, en el que en cada iteración se aplica un desplazamiento a la derecha sobre `in` en i posiciones, extrayendo el bit menos significativo mediante el and binario, aplicando AND 1. De esta forma, conseguimos que `out[i]` adquiera la conversión de decimal a binario.
3. Es imprescindible añadir explícitamente la restricción `out[i] * (out[i] - 1) == 0`. Esta ecuación sí se integra en el sistema de restricciones, asegurando que la señal solo pueda adoptar los valores lógicos 0 o 1. Sin embargo, `out[i] <- (in >> i) & 1` solo asigna y no es añadida al sistema de ecuaciones.
4. Una vez realizadas la asignación y restricción de `out[i]`, se produce la reconstrucción del valor de entrada, este caso el -3. Es en este punto donde la función *Num2Bits()* nos permite determinar si el número es negativo.
5. Como hemos mencionado, al trabajar con un cuerpo finito $\mathbb{F}_p$, el valor -3 se representa como $p-3$, un número positivo extremadamente grande que requiere cerca de 254 bits para ser expresado. Dado que el tamaño máximo permitido es 10, el bucle de reconstrucción solo sumará los pesos de a lo sumo 10 bits.
6. Como consecuencia, el resultado final de la reconstrucción en `lc1` será un valor que nunca igualará al valor de entrada `in` ($p-3$). Al no cumplirse la restricción `lc1 == in`, el circuito resultará insatisfacible, detectando así que el número original no es representable con 10 bits y, por tanto, se comporta como un valor fuera de rango o "negativo".

■

Operadores `==` y `!=`

Los operadores `==` y `!=` plantean una estrategia de implementación basada en dos restricciones claves:

- `(in1-in2) * equal_signal = 0`
- `1 - ((in1-in2) * inv_signal) = equal_signal`

Estas dos ecuaciones son imprescindibles, ya que las usaremos para determinar si `in1 == in2` o `in1 != in2`, utilizando `equal_signal` como indicador.

La señal `equal_signal` será 1 cuando `in1 == in2`, o será 0 cuando `in1 != in2`. Este comportamiento se consigue por medio de las dos ecuaciones descritas anteriormente.

$(in1 - in2) * equal_signal = 0$ es imprescindible para obtener que $equal_signal$ sea 0 cuando $in1$ e $in2$ sean diferentes. Esto se debe a que si $in1$ es distinto a $in2$, entonces el resultado de su diferencia será distinto a 0. Por tanto, la ecuación quedaría así:

$$\begin{aligned} num \cdot equal_signal &= 0 \\ num &\in \mathbb{R} \setminus \{0\} \end{aligned} \tag{4.3}$$

De esta forma, al ser num un valor distinto de cero, la única forma de que se satisfaga la ecuación es forzar a que $equal_signal$ sea obligatoriamente 0. De este modo, la señal será igual a 0, indicando que $in1$ e $in2$ son diferentes.

Observación: Si $in1$ e $in2$ fuesen iguales, entonces su diferencia sería igual a 0, por tanto $equal_aux$ podría tomar cualquier valor e $(in1 - in2) * equal_signal = 0$ seguiría satisfaciéndose:

$$\begin{aligned} 0 \cdot equal_signal &= 0 \\ equal_signal &\in \mathbb{R} \end{aligned} \tag{4.4}$$

Por ello, es importante la segunda ecuación, ya que además de que $equal_signal$ puede ser cualquier valor, podría ser 0 también, indicando erróneamente que $in1$ e $in2$ son diferentes.

Esta segunda ecuación, $1 - ((in1 - in2) * inv_signal) = equal_signal$, además de asegurar que $equal_signal$ sea distinta de cero, garantiza que el resultado sea estrictamente 1 siempre que $in1$ e $in2$ sean idénticos.

Esto se consigue mediante el uso de una segunda señal, inv_signal , la cual garantiza que $equal_signal$ tome el valor 1 cuando $in1$ e $in2$ son iguales. Dado que en este escenario la diferencia entre ambas entradas es nula, cualquier valor de $inv_signal \in \mathbb{R}$ resultará en un producto igual a cero, derivando en el siguiente comportamiento:

$$\begin{aligned} 1 - ((in1 - in2) \cdot inv_signal) &= equal_signal \\ 1 - (0 \cdot inv_signal) &= equal_signal \\ 1 - 0 &= equal_signal \\ 1 &= equal_signal \end{aligned} \tag{4.5}$$

Como puede observarse, aunque en la primera ecuación para $in1 == in2$ la señal $equal_signal$ puede tener cualquier valor, con esta segunda ecuación forzamos a que su valor sea 0, en caso de que $in1 \neq in2$, o bien 1, si $in1 == in2$.

En caso de que $in1 \neq in2$, entonces $equal_signal$ sería 0, obteniendo un comportamiento diferente:

$$\begin{aligned}
1 - ((in1 - in2) \cdot inv_signal) &= equal_signal \\
1 - ((in1 - in2) \cdot inv_signal) &= 0 \\
\text{Como } (in1 - in2) &\in \mathbb{R} \setminus \{0\} \\
inv_signal &= \frac{1}{in1 - in2} \\
1 - 1 &= 0 \\
0 &= 0
\end{aligned} \tag{4.6}$$

Gracias a la señal `inv_signal`, incluso cuando ambas entradas son diferentes, la restricción puede satisfacerse ajustando el valor de la señal al inverso de la diferencia, para así obtener como producto de estas el 1.

Operadores $<, \leq, >$ y $\geq$

Para la implementación de dichos operadores únicamente es necesaria la función *SecureLessThanEq(n)*.

El motivo reside en que una comparación entre una variable a y b , con *SecureLessThanEq(n)* podemos conocer si $a \leq b$ o bien $a > b$ en caso contrario (*GreatherThan()*).

Siguiendo esta lógica, podemos usar la misma función invirtiendo el orden de los parámetros, consiguiendo de esta manera los operadores *GreatherThanEq()* y *LessThan()* si $b \leq a$ o $b > a$.

La implementación de *SecureLessThanEq(n)*, aparte de utilizar la función *Num2Bits()* para asegurar que cada ambas variables no son números negativos, aplicamos sobre ella la operación $2^n - A + B$. Es decir, dadas las variables A y B , aplicamos $-A + B$. La inversión del valor de A y la suma de B es equivalente a la operación $B - A$. Una vez terminada esta operación es transformada a binario para ser evaluada.

Por tanto, cuando el minuendo es mayor o igual que el sustraendo, el resultado de la transformación binaria de la suma $B - A$ produce un acarreo (overflow) en la posición n (el bit extra, bit más significativo). Por otro lado, si el minuendo es menor que el sustraendo, no se produce dicho overflow.

De esta forma sabemos que si el bit de acarreo es 1, podemos concluir que $B \geq A$, ya que al hacer la suma con $2^n - A + B$, tenemos que:

$$\begin{aligned}
&2^n - A + B \\
&B + (2^n - A)
\end{aligned} \tag{4.7}$$

Siguiendo la misma lógica, si el bit de acarreo es 0 significa que A es mayor que B ya que $B - A < 0$, y de esta forma obtendríamos $2^n + (num.negativo) < 2^n$, concluyendo así que no podría producirse un overflow.

Con esta pequeña modificación, al usar para ambos número a a comparar la función *Num2Bits()* nos aseguramos de que se cumpla la restricción de ser número enteros positivos dentro del rango.

4.1.1.4. Operadores de desplazamiento

La operación de desplazamiento implica la transformación de un número determinado al sistema binario y, después de estar en este sistema, desplazar el número binario por una cantidad específica de bits, indicada por el usuario.

Para imitar el comportamiento descrito anteriormente, haremos uso de la función *Num2Bits()* descrita anteriormente para obtener el número a desplazar en binario y poder aplicar las transformaciones diferentes.

Los pasos posteriores a la transformación numérica son dependientes al tipo de desplazamiento. En función de si se trata de un desplazamiento a la izquierda o a la derecha, la construcción del sistema de ecuaciones será diferente.

Por otra parte, es importante destacar que para estos operadores también se aplica el operador `op_(n)` (definido en la **Sección 4.1.1.3**), de forma que el usuario puede limitar el tamaño del valor a desplazar si es conocedor del mismo.

Observación: Hay que asegurar de que el tamaño de N del número a desplazar sea mayor o igual que el número de bits que se quieren desplazar.

Desplazamiento a la izquierda

En este tipo de operación, el objetivo es desplazar un número X de bits (especificados por el usuario) desde las posiciones menos significativas hacia las más significativas; es decir, hacia la izquierda.

Para lograr este comportamiento, los espacios que quedan vacíos en la parte menos significativa son sustituidos por 0.

Esta explicación se complementa con el siguiente ejemplo, que describe el comportamiento de un desplazamiento a la izquierda para la operación $11 \ll 2$:

Ejemplo 4.5: Desplazamiento a la izquierda

- Dado el número decimal 11, lo convertimos a binario: 00001011
- Una vez realizada la transformación a binario, desplazamos 2 bits a la izquierda: 00001011 $\rightarrow$ 00101100
- Al transformarse los últimos dos bits en 0, el objetivo es agregar n ceros al final del array, completando así la representación binaria tras el desplazamiento.



Desplazamiento a la derecha

De manera similar al desplazamiento a la izquierda, el objetivo es desplazar un número X de bits (especificados por el usuario) desde las posiciones más significativas hacia las menos significativas; es decir, hacia la derecha.

Para lograr este comportamiento, los espacios que quedan vacíos en la parte más significativa son sustituidos por 0.

Esta explicación se complementa con el siguiente ejemplo, que describe el comportamiento de un desplazamiento a la derecha para la operación $11 \gg 2$:

- Dado un número decimal, lo convertimos a binario, en este caso 11: 00001011
- Con el número en binario, desplazamos 2 bits a la derecha: 00001011 $\rightarrow$ 00000010
- Como podemos ver, los últimos dos bits se han despreciado, añadiendo dos ceros al principio del número binario.

De la misma forma en la que se ha desarrollado estas operaciones, se definirán las ecuaciones que formarán parte del sistema dentro del compilador para reflejar ambos comportamientos.

Traducción de operadores

Operación	Regla de Transformación
$e = e_1 \ll e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \ll e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} DL(253).in1 = e'_2, \\ DL(253).in2 = e'_1, \\ DL(253).out = v_{aux} \end{array} \right\}, v_{aux} \right)}$
$e = e_1 \gg e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \gg e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} DR(253).in1 = e'_2, \\ DR(253).in2 = e'_1, \\ DR(253).out = v_{aux} \end{array} \right\}, v_{aux} \right)}$
$e = e_1 \ll _ (n) e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \ll _ (n) e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} DL(n).in1 = e'_2, \\ DL(n).in2 = e'_1, \\ DL(n).out = v_{aux} \end{array} \right\}, v_{aux} \right)}$
$e = e_1 \gg _ (n) e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \gg _ (n) e_2) = \left(C_1 \wedge C_2 \wedge \left\{ \begin{array}{l} DR(n).in1 = e'_2, \\ DR(n).in2 = e'_1, \\ DR(n).out = v_{aux} \end{array} \right\}, v_{aux} \right)}$

4.1.2. Casos inductivos: Reglas de transformación de instrucciones

4.1.2.1. Instrucciones condicionales

Como se ha mencionado anteriormente, en Circom cada señal representa un cable del circuito cuyo valor es único. Por esta razón, no pueden cambiar su valor de forma dinámica una vez establecidas, ya que el sistema de restricciones debe ser estático para su correcta verificación.

Para simular decisiones condicionales basadas en entradas, se deben usar expresiones aritméticas que implementen la lógica de selección, de manera que una señal no pueda reasignar su valor.

Esta lógica de selección se basa en combinar los posibles resultados mediante una operación aritmética. En lugar de saltar de una línea de código a otra, el circuito calcula ambos caminos y utiliza la condición como un interruptor encargado de resolver la bifurcación, decidiendo qué valor se asigna finalmente a la señal.

Formalmente, definimos una variable binaria $cond \in \{0, 1\}$, que representa el resultado final obtenido tras la evaluación de la condición **cond**. La selección de valores dentro de un bloque condicional se formaliza aplicando restricciones aritméticas sobre cada asignación de variables contenida en dichos bloques.

Si dentro del bloque **if** (al que llamaremos S_1) se realiza una asignación de la forma $v_{aux} = \text{valor}_{if}$, esta se define mediante el siguiente sistema de restricciones:

$$(v_{aux} - \text{valor}_{if}) * cond = 0 \quad (4.8)$$

De manera análoga, si se presentase una instrucción condicional con estructura **if-else**, para cada par de asignaciones equivalentes sobre una misma variable v_{aux} en los bloques de instrucciones S_1 (donde $v_{aux} = \text{valor}_{if}$) y S_2 (donde $v_{aux} = \text{valor}_{else}$), el sistema de restricciones generado sería el siguiente:

$$(v_{aux} - \text{valor}_{if}) * cond = 0 \quad (4.9)$$

$$(v_{aux} - \text{valor}_{else}) * (1 - cond) = 0 \quad (4.10)$$

Esta estructura garantiza que:

- Si $cond = 1$, el término $(1 - cond)$ se anula, activando la primera restricción y resultando en $v_{aux} = \text{valor}_{if}$.
- Si $cond = 0$, el término $cond$ se anula, activando la segunda restricción y resultando en $v_{aux} = \text{valor}_{else}$.

La idea es partir de este caso baso para implementar las estructuras condicionales anidadas de forma inductiva. De esta forma, para ejecutar un determinado bloque de restricciones, todos las condiciones lógicas deben cumplirse.

En términos prácticos, esto significa que la variable de activación de un bloque anidado es el producto de su propia condición por la variable de control del nivel superior. De esta manera, cualquier camino lógico dentro de una jerarquía se define mediante una cadena de productos de las condiciones ($cond_i$) o sus inversas ($1 - cond_i$), dependiendo de la ruta tomada, donde i es el nivel n de anidamiento.

$$v_{activacion} = v_{cond-1} * v_{cond-2} * \dots * v_{cond-n} \text{ donde } v_{cond-i} \in \{cond_i, 1 - cond_i\}.$$

Este enfoque permite que el compilador transforme cualquier árbol de decisiones, sin importar su profundidad, en una combinación de señales donde cada camino lógico tiene asignado un filtro exclusivo basado en el cumplimiento de todas sus condiciones

precedentes. Así, el sistema de restricciones asociado a un bloque condicional se aplicará cuando el producto de todas sus condiciones precedentes sea igual a uno.

El producto de todas estas condiciones no se puede realizar directamente en una sola restricción. Dado que los sistemas R1CS limitan las restricciones a expresiones cuadráticas, es necesario introducir variables auxiliares intermedias. De este modo, se descompone la operación en una secuencia de productos binarios, garantizando que el circuito sea válido.

Un ejemplo específico de conversión de un bloque condicional a su equivalente utilizando restricciones en Circom sería el siguiente:

Programación Imperativa	Programación Declarativa
<pre> signal input entrada; signal output salida; if(entrada == 0){ salida <-- 1; } </pre>	<pre> signal input entrada; signal output salida; signal is_zero_variable; is_zero_variable <-- entrada == 0; (salida - 1) * is_zero_variable == 0; </pre>

Ejemplo 4.6: Comparación entre bloque condicional imperativo y declarativo ■

El ejemplo anterior ilustra el proceso de transformación de una expresión condicional a una expresión aritmética equivalente, evitando reasignar la señal de salida.

De la misma forma expuesta en el ejemplo, definiremos la inducción partiendo del caso base.

Caso base: if-else

```

1      // Código en Circom haciendo uso de nuestra nueva
      funcionalidad
2      signal input age;
3      signal output out;
4
5      if (age < 0) {
6          out <-- 0;
7      } else {
8          out <-- 1;
9      }

```

```
1      // Codigo en Circom
2      signal input age;
3      signal output out;
4      signal is_less_than_zero;
5
6      // Evaluamos la condicion de forma directa
7      is_less_than_zero <-- age < 0;
8
9      // Restricciones para cada asignacion:
10     (out - 0) * is_less_than_zero === 0;
11     (out - 1) * (1 - is_less_than_zero) === 0;
```



Ejemplo 4.7: Caso base condicional

El patrón de traducción identificado es el siguiente:

1. Crear una componente equivalente a la expresión lógica utilizada en el `if`
2. `if`: La restricción se activa cuando el resultado de dicho componente es igual a 1
3. `else`: La restricción se activa mediante la negación lógica de la expresión anterior

Este ejemplo representa el caso base, el cual se aplica de manera inductiva para estructuras condicionales más complejas.

La traducción formal de este caso es la siguiente:

Caso inductivo: Condicionales anidados

Partiendo del caso base, es trivial aplicar la inducción para alcanzar la transformación de bloques condicionales complejos en sistemas de ecuaciones.

A continuación, un ejemplo guiado donde aplicamos las hipótesis del caso base para llegar al resultado final.

Ejemplo 4.8: Caso inductivo condicional

```
1  signal input age;
2  signal input out;
3
4  if (age < 20) {
5      if(age < 10){
6          out <-- 1;
7      }
8      else{
9          out <-- 2;
10     }
11 } else if(age < 40){
12     if(age < 30){
13         out <-- 3;
14     }
15     else {
16         out <-- 4;
17     }
18 }
19 else{
20     out <-- 5;
21 }
```

- Por cada condición de igualdad o desigualdad, debe crearse una variable encargada de calcular la condición correspondiente.

```
1  signal input age;
2  signal input out;
3  signal lt_10 <-- age < 10;
4  signal lt_20 <-- age < 20;
5  signal lt_30 <-- age < 30;
6  signal lt_40 <-- age < 40;
```

- Partiendo de lo expuesto en los apartados anteriores, las condiciones excluyentes de cada bloque condicional actúan como selectores, permitiendo que la señal `out` tome el valor correcto en función del camino ejecutado.

```
1  signal lt_20_10 <== lt_10 * lt_20;
2  signal lt_20_not_10 <== (1 - lt_10) * lt_20;
3  signal rama_else_if <== lt_40 * (1 - lt_20);
4  signal lt_40_30 <== lt_30 * rama_else_if;
5  signal lt_40_not_30 <== (1 - lt_30) * rama_else_if;
6  signal lt_not_20_not_40 <== (1 - lt_20) * (1 - lt_40);
7
8  (out - 1) * lt_20_10 == 0;
9  (out - 2) * lt_20_not_10 == 0;
10 (out - 3) * lt_40_30 == 0;
11 (out - 4) * lt_40_not_30 == 0;
12 (out - 5) * lt_not_20_not_40 == 0;
```

- El uso de las señales intermedias `rama_else_if` es estrictamente necesario para asegurar que la construcción de los productos resulte en una operación cuadrática, evitando así ecuaciones de grado superior.



Aplicando la inducción, se repite el mismo patrón en el que utiliza la condición actual y la negación de sus anteriores, construyendo de esta forma las expresiones condicionales necesarias para representar correctamente en el sistema de ecuaciones el acceso a cada bloque condicional.

De esta manera, podemos transformar cualquier bloque condicional en un sistema de ecuaciones equivalente, cumpliendo con la naturaleza estática de Circom.

Traducción de operadores

Operación	Regla de Transformación
$\text{if } cond \text{ then } S_1 \\ \text{else } S_2$	$\frac{TC(cond) = (C_{cond}, cond') \quad TC(S_1) = (C_1, S'_1) \quad TC(S_2) = (C_S, S'_2)}{TC(\text{if } cond \text{ then } S_1 \text{ else } S_2) = (C_{cond} \wedge C_1 \wedge C_S \wedge \{v_{aux} = cond' \cdot S'_1 + (1 - cond') \cdot S'_2\}, v_{aux})}$

Implementación del problema

Antes de explicar cómo se ha llevado a la práctica la implementación, es necesaria una visión general sobre cómo está construido el compilador de Circom.

El compilador de Circom está escrito en Rust, un lenguaje de programación que destaca por su rapidez y gestión eficiente de memoria [18]. Estas características hacen que sea un gran candidato para la construcción de compiladores, ya que en esta tarea se requiere procesar y modificar estructuras de datos de manera eficiente y sin errores.

La pieza central sobre la que opera cualquier compilador es la estructura de datos conocida como árbol de sintaxis abstracta o AST.

Definición 8 *Un árbol de sintaxis abstracta es una representación simplificada del código fuente escrito en un lenguaje de programación dado, donde cada nodo del árbol denota una construcción del lenguaje que ocurre en el código fuente. [19].*

Una construcción del lenguaje podría ser una declaración, una expresión, un operador, etc.

El compilador recorre este árbol para analizar, validar y transformar el programa, convirtiéndolo en su representación final, en el caso de Circom, un sistema de restricciones en formato R1CS.

En este contexto, se ha implementado la cabecera `autocomplete`. Esta cabecera es el indicador que le comunica al compilador si el código tiene que ser procesado por medio del código base del lenguaje o mediante la nueva funcionalidad implementada (como puede verse en el **Ejemplo 1.2**).

De esta forma, durante la construcción del AST, al identificar la presencia de dicha cabecera, el compilador genera los nodos correspondientes haciendo uso del código añadido para cada operador, en lugar del código de compilación habitual.

Para cada operador se han implementado sus respectivas reglas de traducción, para así asegurar que las restricciones añadidas al sistema de ecuaciones son las correctas para simular su lógica.

Como hemos mencionado, cada operador es un posible nodo del AST. A continuación se expone un ejemplo de cómo el compilador construiría el árbol en función del siguiente código.

Figura 5.1: Construcción de AST

CÓDIGO	ÁRBOL AST
<pre> 1 template Calculo() { 2 signal input a; 3 signal input b; 4 signal output c; 5 c <-- (a/b) + ((b !=0) * 5); 6 } 7 component main = Calculo(); </pre>	

Cada nodo tiene su implementación dentro del compilador, que corresponde con las reglas de traducción especificadas en el capítulo anterior para cada operador.

Debido al tamaño que ocuparía la explicación de las implementaciones de cada uno de los operadores, explicaremos únicamente la implementación de uno de ellos, dejando como referencia el repositorio con la implementación si fuera preciso consultarlo [20].

5.1. Ejemplo de implementación

El operador elegido como ejemplo ilustrativo es el de desplazamiento a la izquierda. Como fue explicado anteriormente en la **Sección 4.1.1.4**, para este operador se siguió la siguiente lógica para simular su comportamiento:

1. Transformar el número decimal a desplazar a sistema binario
2. Mover los k bits menos significativos a la izquierda un número de k posiciones.
3. Los k bits menos significativos sustituirlos por cero.

A la hora de realizar la conversión a binario tenemos que realizar una comprobación previa sobre el tamaño del desplazamiento.

Tal y como se explicó en la **Sección 4.1.1.3** y se definió en la **Sección 4.1.1.4**, si el usuario conoce el tamaño de los valores con los que va a trabajar puede utilizar el $\ll_n$ sobre los operadores de desplazamientos, de forma que podrá acotar el tamaño de los operandos utilizando esta utilidad.

Por tanto, si el tamaño de los valores a desplazar es conocido, el usuario podría limitar el tamaño, ya que por defecto la funcionalidad utiliza el tamaño máximo correspondiente a 253, lo que implica muchas más restricciones de las necesarias.

Por ejemplo, si el usuario conoce que el tamaño de los valores con los que va a trabajar es igual o inferior a cuatro, podría aplicar esta limitación tal que así: `11 <<_(4) 2`.

Debido a este modo en el que el tamaño puede ser limitado, tiene que realizarse la comprobación de que el número n elegido como límite y el número k de bits a desplazar no presente una situación en la que $k > n$.

Esta verificación previa es necesaria, ya que no puede desplazarse un número cuyo tamaño es menor que el del desplazamiento. Por este motivo, esta comprobación antes de realizar la implementación es necesaria.

```

1      let k = get_constant_usize(r_value)
2      .expect("Shift amount must be a constant usize");
3      assert!(k <= n, "Shift exceeds bit width");
4
5
6      let mut constraints = vec![];
7      let mut new_vars_name = vec![];
8      let mut b_signals: Vec<AExpr> = vec![];
```

Una vez realizada la comprobación, si se cumple la condición $k \leq n$ el código prosigue su curso, declarando las restricciones y señales necesarias para la implementación.

Como fue especificado, el primer paso en esta lógica es la conversión decimal a binaria del número a desplazar. Para ello, hacemos uso de la función `decimal_to_bits()` implementada como método general destinado a las conversiones y así reutilizar código. Puede consultarse en el repositorio del proyecto [20].

En resumen, como su nombre indica, esta función tiene como objetivo la conversión de números en sistema decimal a sistema binario. De esta forma, durante la conversión crea las señales necesarias y restricciones, estas últimas añadidas al conjunto global de restricciones que serán incorporadas al sistema de ecuaciones del operador.

Tras finalizar la función, se obtiene como resultado un array de tamaño n donde cada posición es un dígito binario, representando de esta forma el número binario equivalente.

```

1      // field_copy corresponde con el numero primo del cuerpo finito
2      // Transformar numero a desplazar en binario
3      let (b_signals, constraints_bit) = decimal_to_bits(
4      &l_value, n, &field_copy, runtime, &mut new_vars_name
5      );
6      constraints.extend(constraints_bit);
```

Una vez tenemos el número binario, podemos aplicar las restricciones correspondientes, pero antes de ello necesitamos declarar las señales intermedias que vamos a necesitar para forzar la asignación de los k bits menos significativos a cero y los bits entre $[k, n]$ al número entre $[0, n-k]$.

```

1
2      let mut s_signals = vec![];
3
4      // 2. Crear senales intermedias para asignar cada bit a un valor
5      for _ in 0..n {
6          let shift_signal_name = format!("s_{}", runtime.new_added_vars);
```

```

7         runtime.new_added_vars += 1;
8         s_signals.push(AExpr::Signal {
9             symbol: shift_signal_name.clone()
10        });
11        new_vars_name.push(shift_signal_name);
12    }

```

Con las señales intermedias declaradas, podemos forzarlas para así conseguir el desplazamiento del número binario.

Siguiendo el ejemplo utilizado en la **Sección 4.1.1.4** para explicar este operador, el número binario 1011 con un desplazamiento de $k = 2$ en este punto se transformaría a 1111.

Esto se debe a que primeramente realizamos el desplazamiento de los k bits menos significativos a las posiciones entre $[k,n)$.

```

1         // 3. Forzar el desplazamiento. Movemos los k bits a la izquierda
2         for i in k..n {
3             let eq = AExpr::sub(
4                 &s_signals[i], &b_signals[i-k], &field_copy
5             );
6             let c = AExpr::transform_expression_to_constraint_form(
7                 eq, &field_copy
8             ).unwrap();
9             constraints.push(c);
10        }

```

Para completar el desplazamiento, queda forzar a las señales entre $[0,k)$ a que su valor sea cero, de la misma forma que en el caso anterior forzamos a que las señales entre $[k,n)$ fuesen el número binario entre $[0,n-k)$.

De esta forma, el número binario desplazado pasaría de 1111 a 1100.

```

1         // 4. Los k bits menos significativos = 0
2         for i in 0..k {
3             let eq = AExpr::sub(
4                 &s_signals[i],
5                 &AExpr::Number { value: BigInt::from(0) },
6                 &field_copy
7             );
8             let c = AExpr::transform_expression_to_constraint_form(
9                 eq, &field_copy
10            ).unwrap();
11            constraints.push(c);
12        }

```

Una vez finalizada toda la lógica correspondiente al desplazamiento a la izquierda, el paso final es reconstruir el número binario desplazado en decimal.

La reconstrucción de un número binario a decimal se realiza a través del **método de cambio de base**. De forma que se realiza la suma de los productos entre cada dígito y potencia de base dos según el índice de posición [21].

Siguiendo este método, recorreremos el número binario desplazado aplicando para cada dígito el producto del mismo con su potencia de base dos en función del índice que ocupa. Tras completar el recorrido del número aplicando este método a cada uno de los dígitos, la variable `sum_out` acumula la suma de estos productos, siendo la variable que almacena el resultado de la conversión binaria a decimal.

```

1      let mut sum_out = AExpr::Number { value: BigInt::from(0) };
2      for i in 0..n {
3          let power_of_two = BigInt::from(1) << i;
4          let sum = AExpr::mul(
5              &s_signals[i],
6              &AExpr::Number { value: BigInt::from(power_of_two) },
7              &field_copy
8          );
9          sum_out = AExpr::add(&sum_out, &sum, &field_copy);
10     }
11
12     Ok((sum_out, constraints, new_vars_name))

```

De esta forma, aplicando estos bloques de código en el mismo orden en el que han sido expuestos se consigue plasmar la lógica de este operador aplicando restricciones que formarán parte del sistema de ecuaciones, consiguiendo de esta forma simular completamente la lógica deseada.

Para el resto de operadores se aplica la misma lógica pero siguiendo las reglas de traducción especificadas. De hecho, muchas funciones como los operadores **Igual** y **Distinto** presentan la misma lógica con lo que puede abstraerse mucho código.

En resumen, siguiendo esta vía se transita desde una idea hasta un sistema más útil, logrando unir la parte teórica de las reglas de traducción con la parte práctica destinada a la implementación de estas reglas, consiguiendo que estos operadores funcionen según lo esperado.

Verificación de los operadores

La verificación de operadores es un paso fundamental en este trabajo. Este proceso es necesario para garantizar que la nueva funcionalidad implementada construye sistemas de ecuaciones equivalentes a los generados naturalmente por el lenguaje Circom.

Para realizar esta verificación de forma segura y fiable se ha empleado la herramienta *ZK-GENVER*, desarrollada por el grupo de investigación Costa [15].

Definición 9 *ZK-GENVER es una herramienta diseñada para la verificación de sistemas de ecuaciones en formato R1CS.*

Esta herramienta ofrece tres funcionalidades principales. En primer lugar, la verificación determinista del sistema de ecuaciones, comprobando que para cada entrada produce una única salida. En segundo lugar, la generación de archivos `smt2`, que contienen las ecuaciones del circuito, las cuales pueden ser alteradas manualmente para así asignar valores de entrada específicos con el fin de consultar la salida correspondiente y validar así que la lógica es la esperada. Por último, la comparación de equivalencia entre dos archivos R1CS, transformándolos a archivos `smt2`, lo que posibilita contrastar el código base de Circom con el que incorpora la nueva funcionalidad.

Por tanto, para llevar a cabo una verificación exhaustiva de cada operador implementado, se emplearán las tres funcionalidades que *ZK-GENVER* ofrece, siguiendo el mismo orden en el que han sido expuestas:

1. Verificación determinista
2. Validación de la lógica asignando valores a las entradas
3. Comparación de equivalencia entre los archivos `smt2` generados

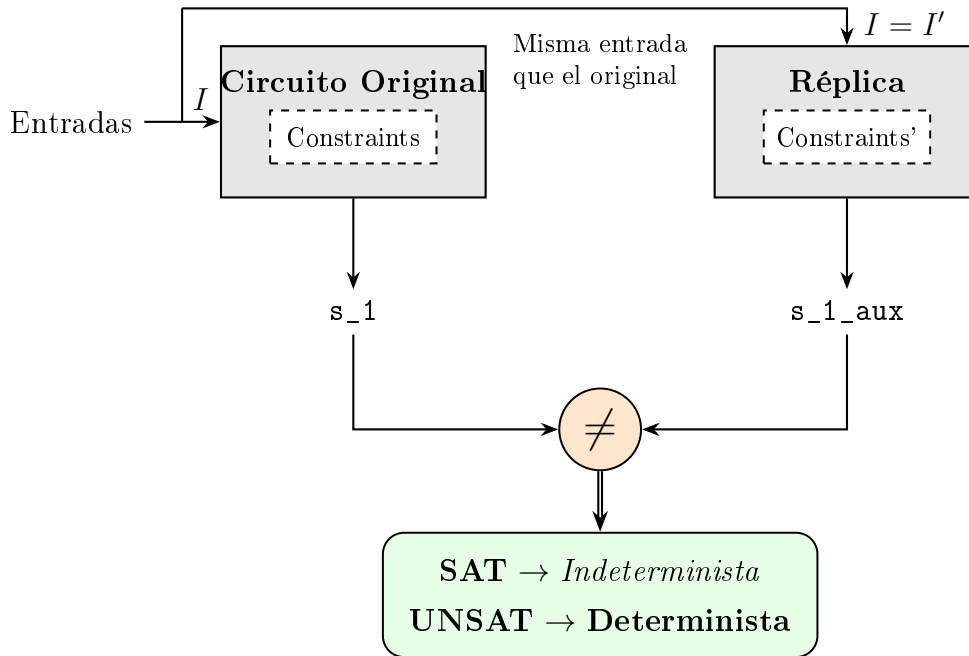
ZK-GENVER realiza la verificación determinista a partir de los archivos `smt2`, los cuales son generados automáticamente por la herramienta tomando como base los archivos R1CS que contienen todas las restricciones del programa original. Este archivo `smt2` es pasado al *resolutor SMT*.

Definición 10 *Un resolutor SMT es una herramienta capaz de determinar si un sistema de ecuaciones es satisfactible, es decir, si existe al menos una asignación de valores que lo satisfaga, o insatisfactible, en caso de que no exista ninguna.*

La comprobación se centra en la imposición de la restricción $s_1 \neq s_1_aux$. Mediante este esquema, el sistema compara la salida del circuito original con la de una réplica exacta que cuanta con las mismas entradas, forzando al *solver* a buscar si existe alguna solución en la que para las mismas entradas produzca valores de salida distintos.

El resultado de esta búsqueda puede ser SAT o UNSAT. Si el sistema es **insatisfactible (UNSAT)**, se garantiza que el circuito es determinista, ya que no existe ninguna solución en la que para las mismas entradas haya salidas diferentes. Por otro lado, si el sistema es **satisfactible (SAT)**, el *solver* halla un contraejemplo que evidencia un comportamiento indeterminista. Esto revela que el circuito permite diferentes valores de salida para los mismos valores de entrada, señalando así una vulnerabilidad en la la lógica del sistema.

Figura 6.1: Esquema de la comprobación de unicidad determinista.



Este principio es la base sobre la que *ZK-GENVER* construye sus verificaciones. En lugar de probar múltiples entradas, lo cual sería inviable, la herramienta añade al final del archivo `smt2` la restricción `(assert (not (= s_1 s_1_aux)))` (resaltada en azul en el siguiente código), consultando al solver si el sistema resultante es satisfactible o no.

Ejemplo 6.1: Archivo smt2 generado por el operador Igual

```
1      (set-logic QF_FF)
2      (define-sort FF0 () (_ FiniteField p)) ; p = primo del campo, omitido por
        legibilidad
3      (declare-fun s_1 () FF0)
4      (declare-fun s_1_aux () FF0)
5      (declare-fun s_2 () FF0)
6      (declare-fun s_3 () FF0)
7      (declare-fun s_4 () FF0)
8      (declare-fun s_4_aux () FF0)
9      (declare-fun s_5 () FF0)
10     (declare-fun s_5_aux () FF0)
11     (assert (= (as ff0 FF0) (ff.mul (ff.add (ff.mul s_3 (as -1)) s_2) s_4)))
12     (assert (= (as ff0 FF0) (ff.mul (ff.add (ff.mul s_3 (as -1)) s_2) s_4_aux)))
13     (assert (= (as ff0 FF0) (ff.add (ff.mul (ff.add (ff.mul s_2 (as -1 FF0)) s_3)
        s_5) (ff.add (as -1 FF0) s_4))))
14     (assert (= (as ff0 FF0) (ff.add (ff.mul (ff.add (ff.mul s_2 (as -1 FF0)) s_3)
        s_5_aux) (ff.add (as -1 FF0) s_4_aux))))
15     (assert (= (as ff0 FF0) (ff.add (ff.mul s_1 (as -1 FF0)) s_4)))
16     (assert (= (as ff0 FF0) (ff.add (ff.mul s_1_aux (as -1)) s_4_aux)))
17     (assert (not (= s_1 s_1_aux)))
18     (check-sat)
```

Si el solver devuelve UNSAT, se concluye que no existe ninguna combinación de entradas que produzca salidas distintas, confirmando así el determinismo del operador.

De esta forma, *ZK-GENVER* es capaz de verificar de manera formal si un sistema de ecuaciones es determinista.

En cuanto a la segunda funcionalidad, se parte del mismo archivo `smt2` generado en la verificación determinista, el cual contiene las ecuaciones del sistema de restricciones. De esta forma, es posible modificar este archivo manualmente para asignar valores a las entradas, haciendo posible consultar la salida esperada y comprobar así que la lógica del operador es la correcta en función de las entradas asignadas.

Partiendo del **Ejemplo 6.1**, habría que modificar el archivo `smt2` para asignar los valores deseados a las señales de entradas.

Ejemplo 6.2: Asignación de valores en archivo smt2

```
1      (set-logic QF_FF)
2      (define-sort FF0 () (_ FiniteField p)) ; p = primo del campo, omitido por
        legibilidad
3      (declare-fun s_1 () FF0)
4      (declare-fun s_1_aux () FF0)
5      (declare-fun s_2 () FF0)
6      (declare-fun s_3 () FF0)
7      ...
8      (assert (= s_2 3))
9      (assert (= s_3 2))
10     (check-sat)
```

Como puede apreciarse, además de añadir dos restricciones para asignar valores a las señales de entrada `s_2` y `s_3`, la restricción `(assert (not (= s_1 s_1_aux)))` encargada de verificar el determinismo del sistema es suprimida.

El motivo de esta supresión es que llegados a este paso, no buscamos verificar si nuestro sistema es determinista o no, sino concluir si el sistema de ecuaciones generado por nuestra nueva funcionalidad realmente describe la lógica esperada.

Una vez realizada la asignación de las señales de entrada y procesado el archivo `smt2` modificado, el *solver* determina el valor resultante para la señal de salida, que en este caso corresponde a `s_1`.

De esta forma, podemos concluir si nuestro sistema de ecuaciones, además de ser determinista, realmente representa la lógica adecuada.

Una vez verificada la lógica y el determinismo del sistema, el paso final consiste en validar su equivalencia con los circuitos originales escritos en Circom. Dado que estos últimos carecen de la nueva funcionalidad, actúan como un referente de veracidad, permitiéndonos confirmar que el nuevo diseño mantiene el comportamiento correcto y determinista de las implementaciones en Circom sin utilizar nuestra funcionalidad.

De este modo, la herramienta utiliza los archivos `smt2` para contrastar las restricciones generadas por la nueva funcionalidad frente a las de la librería `circomlib` [16]. Al emplear operadores estándar cuya fiabilidad ya es conocida, se garantiza que la nueva implementación mantiene la equivalencia funcional con los métodos tradicionales de Circom.

De esta forma, si ambos sistemas de restricciones son equivalentes, obtenemos una confirmación adicional de que la nueva funcionalidad describe correctamente la lógica esperada, reforzando así los resultados obtenidos previamente mediante la validación con los archivos `smt2` de cada operador.

A continuación se presentan los resultados obtenidos para cada operador siguiendo el proceso de verificación descrito.

6.1. Resultados obtenidos

6.1.1. Operadores Aritméticos

Dentro del conjunto de operadores aritméticos, se han obtenido los siguientes resultados tras la verificación.

Operación	Resultado obtenido
Suma	Verificado exitosamente ✓
Resta	Verificado exitosamente ✓
Multiplicación	Verificado exitosamente ✓
División	Timeout (Usando implementación propia equivalente B.1.1)

Módulo	Timeout (Usando implementación propia equivalente B.1.2)
--------	---

6.1.2. Operadores Lógicos

La verificación de los operadores lógicos es sencilla. La librería `circomlib` dispone de un sección exclusiva de puertas lógicas como `NOT`, `AND` y `OR`, las cuales utilizamos para simular el comportamiento de este tipo de operadores [17].

Observación: La verificaciones se realizaran sobre los códigos a comparar con tamaño $n = 1$. El motivo de este tamaño es generar el número mínimo de restricciones necesarias, ya que al exceder un determinado límite, *ZK-GENVER* no podría verificarlo correctamente, obteniendo como resultado **timeout**

Operación	Resultado obtenido
NOT	Verificado exitosamente ✓
AND	Verificado exitosamente ✓
OR	Verificado exitosamente ✓
AND Binario	Verificado exitosamente ✓ (Usando implementación propia equivalente B.2.1)
OR Binario	Verificado exitosamente ✓ (Usando implementación propia equivalente B.2.2)

6.1.3. Operadores de Comparación

Todos los operadores de comparación han sido verificados satisfactoriamente, confirmando su equivalencia con los operadores de la `circomlib`.

Operación	Resultado obtenido
Igual	Verificado exitosamente ✓
Distinto	Verificado exitosamente ✓
Menor	Verificado exitosamente ✓
Menor igual	Verificado exitosamente ✓
Mayor	Verificado exitosamente ✓

Mayor igual	Verificado exitosamente ✓
-------------	---------------------------

6.1.4. Operadores de Desplazamiento

Ambos tipos de desplazamiento han sido sometidos a las tres pruebas de *ZK-GENVER*, concluyéndose que son deterministas y que sus sistemas de restricciones son equivalentes.

Operación	Resultado obtenido
Desplazamiento a la izquierda	Verificado exitosamente ✓ (Usando implementación propia equivalente B.3.1)
Desplazamiento a la derecha	Verificado exitosamente ✓ (Usando implementación propia equivalente B.3.2)

CAPÍTULO 7

Experimentos

7.1. Aplicación de la generación automática sobre juegos simples

Los juegos simples sobre los que se han realizado estos experimentos corresponden a *Piedra, Papel y Tijera*, *Caníbales y misioneros*, *Tres en raya* y el *Sudoku*.

Las implementaciones pueden ser consultadas en su respectivo repositorio, donde también podrán verse los archivos `input.json` y `witness.json` [11].

7.1.1. Piedra, Papel y Tijera

La lógica de este juego fue explicada en la sección **Ejemplo: Piedra, Papel y Tijera**.

Para comprobar que la lógica de ambas implementaciones es correcta y equivalente, utilizaremos el archivo `witness.json` generado al compilar cada una de ellas.

Como se mencionó en el primer capítulo, este archivo calcula los valores de todas las señales del circuito. De esta manera, podemos utilizar un archivo `input.json` para asignar valores específicos a las señales de entrada del programa.

Así, una vez configuradas las entradas, podemos aplicar dicho archivo sobre el ejecutable del programa y estudiar el `witness.json` resultante.

Por tanto, al contener el valor de todas las señales, es posible verificar si el resultado de la señal de salida del programa es el esperado.

Para realizar esta comprobación, el archivo `input.json` utilizado para estudiar el valor que toman las señales es el siguiente:

```
1  {
2    "jugador1": 1,
3    "jugador2": 2
4  }
```

A la hora de implementar el circuito, las señales de entrada `jugador1` y `jugador2` representan la elección de cada jugador, donde el número 0 representa la piedra, el

1 el papel y el 2 la tijera. Por otro lado, para la señal de salida **ganador**, el valor 0 indica la victoria del jugador 1, el 1 la del jugador 2 y el 2 representa un empate.

De esta manera, acorde a esta asignación de valores, el jugador 1 ha sacado papel y el jugador 2 tijera, por tanto el resultado de la señal **ganador** debería ser 1, indicando que el jugador 2 ha ganado.

Tras aplicar dichos valores a las señales de entrada y estudiar el **witness.json**, el valor de la señal **ganador** es 1 para ambas implementaciones.

Del mismo modo, se modificaron los valores de **jugador1** y **jugador2** en el archivo de entrada para comprobar los escenarios correspondientes a **ganador** = 0 y **ganador** = 2, concluyendo así que la lógica es correcta para las tres situaciones posibles.

Por otro lado, en cuanto al número de restricciones lineales y no lineales generadas por ambos códigos.

Para ello, durante la compilación utilizaremos los niveles de optimización O0 y O2, correspondientes a la optimización desactivada y al nivel máximo respectivamente, obteniendo los siguientes resultados:

Implementación	Número de restricciones O0	Número de restricciones O2
Piedra, Papel y Tijera sin autocomplete	No lineales: 22 Lineales: 41	No lineales: 22 Lineales: 0
Piedra, Papel y Tijera con autocomplete	No lineales: 64 Lineales: 0	No lineales: 64 Lineales: 0

Tras realizar estas compilaciones, se observa que, en ambos niveles de optimización, la implementación sin **autocomplete** es más eficiente que la versión con **autocomplete**, ya que genera un menor número de restricciones no lineales.

7.1.2. Caníbales y misioneros

La lógica de esta implementación se basa en verificar que la secuencia de pasos cumple las reglas del acertijo. Para ello, el circuito comprueba en cada estado tres reglas:

1. El barco debe alternar su posición entre ambas orillas en cada turno.
2. No puede haber una mayoría de caníbales sobre misioneros en ninguna de las orillas para evitar que estos últimos sean devorados.
3. El barco debe transportar obligatoriamente entre una y dos personas como máximo en cada viaje.

El incumplimiento de cualquiera de estas restricciones en al menos un estado resultaría en una solución incorrecta.

Al igual que hicimos en el experimento anterior, utilizaremos el archivo de salida del **witness** para comprobar que todo funciona. Para ello, definimos en el **input.json** la siguiente secuencia de estados:

```
1      {
2          "estados": [
3              [3, 3, 1],
4              [3, 1, 0],
5              [3, 2, 1],
6              [3, 0, 0],
7              [3, 1, 1],
8              [1, 1, 0],
9              [2, 2, 1],
10             [0, 2, 0],
11             [0, 3, 1],
12             [0, 1, 0],
13             [0, 2, 1],
14             [0, 0, 0]
15         ]
16     }
```

A la hora de diseñar el circuito, se ha optado por un enfoque basado en un autómata. La señal de entrada **estados[12][3]** representa la secuencia de estados por los que pasan los personajes para cruzar el río.

Cada uno de los 12 estados contiene tres valores correspondientes al número de misioneros, al número de caníbales y a la posición de la barca (donde 1 representa la orilla inicial y 0 la orilla final).

Por otro lado, para la señal de salida **salida**, el valor 1 indica que la secuencia de movimientos sigue las reglas del juego, mientras que el valor 0 señala que se ha cometido una infracción y, por tanto, la solución propuesta no es la correcta.

De esta manera, acorde a esta asignación de valores, la secuencia introducida representa la solución al acertijo, cualquier modificación sobre esta configuración resultaría en **salida = 0**. Por tanto, el resultado para la actual configuración debería ser **salida = 1**.

Tras aplicar dichos valores a las señales de entrada y estudiar el **witness.json**, el valor de la señal de salida es 1 para ambas implementaciones.

Del mismo modo, se modificaron los valores de la matriz **estados** en el archivo de entrada para comprobar que la salida es igual a 0 al utilizar configuraciones de estados diferentes a la expuesta, concluyendo así que la lógica es correcta para las distintas situaciones posibles.

Por otro lado, en cuanto al número de restricciones lineales y no lineales generadas por ambos códigos.

Para ello, evaluaremos de nuevo el comportamiento del circuito bajo los niveles de optimización -00 y -02, obteniendo los siguientes resultados:

Implementación	Número de restricciones O0	Número de restricciones O2
Caníbales y Misioneros sin <code>autocomplete</code>	No lineales: 602 Lineales: 865	No lineales: 600 Lineales: 0
Caníbales y Misioneros con <code>autocomplete</code>	No lineales: 1511 Lineales: 654	No lineales: 1496 Lineales: 0

Tras realizar estas compilaciones, se observa que, en ambos niveles de optimización, la implementación sin `autocomplete` es más eficiente que la versión con `autocomplete`, ya que genera un menor número de restricciones no lineales.

7.1.3. Tres en raya

La lógica se basa en que uno de los dos jugadores consiga alinear 3 de sus símbolos (cruz o círculo), ya sea en fila, columna o diagonal, o bien ninguno lo consiga y se de un empate.

Si se cumple alguna de estas condiciones, el circuito determina quién es el ganador. De lo contrario, detectará si la partida ha terminado en empate.

Al igual que hicimos en los experimentos anteriores, utilizaremos el archivo de salida del `witness` para comprobar que todo funciona. Para ello, definimos en el `input.json` la siguiente configuración para el tablero:

```

1      {
2          "tablero": [
3              [0, 1, 2],
4              [2, 0, 1],
5              [2, 2, 0]
6          ]
7      }
```

La señal de entrada `tablero[n][n]` representa el estado actual de la partida, donde el número 0 representa los círculos, el 1 las cruces y el 2 indica que la casilla está vacía. Por otro lado, para la señal de salida `ganador`, el valor asignado será 1 si ganan los círculos, el 2 si ganan las cruces y el 0 en caso de empate.

De esta manera, acorde a esta asignación de valores, la secuencia introducida en el ejemplo muestra una línea diagonal completa de círculos, por tanto el resultado para la actual configuración debería ser `ganador = 1`.

Tras aplicar dichos valores a las señales de entrada y estudiar el `witness.json`, el valor de la señal de salida refleja el resultado esperado para ambas implementaciones.

Del mismo modo, se modificaron los valores de la matriz `tablero` en el archivo de entrada para comprobar que la señal toma los valores correctos en escenarios con

un ganador diferente o en situaciones de empate, concluyendo así que la lógica es correcta para las distintas situaciones posibles.

Por otro lado, en cuanto al número de restricciones lineales y no lineales generadas por ambos códigos.

Para ello, evaluaremos de nuevo el comportamiento del circuito bajo los niveles de optimización -O0 y -O2, obteniendo los siguientes resultados:

Implementación	Número de restricciones O0	Número de restricciones O2
Tres en raya sin <code>autocomplete</code>	No lineales: 128 Lineales: 459	No lineales: 128 Lineales: 0
Tres en raya con <code>autocomplete</code>	No lineales: 260 Lineales: 26	No lineales: 260 Lineales: 0

Tras realizar estas compilaciones, se observa que, en ambos niveles de optimización, la implementación sin `autocomplete` es más eficiente que la versión con `autocomplete`, ya que genera un menor número de restricciones no lineales.

7.1.4. Sudoku

La lógica de esta implementación se basa en verificar que la matriz del tablero represente una solución válida según las reglas del juego. Para ello, el circuito comprueba tres condiciones:

1. Cada una de las 9 filas debe contener los números del 1 al 9 exactamente una vez.
2. Cada una de las 9 columnas debe contener los números del 1 al 9 exactamente una vez.
3. Cada una de las 9 subtableros de 3×3 debe contener los números del 1 al 9 exactamente una vez.

El incumplimiento de cualquiera de estas restricciones resultaría en una solución incorrecta.

Al igual que hicimos en los experimentos anteriores, utilizaremos el archivo de salida del `witness` para comprobar que todo funciona. Para ello, definimos en el `input.json` la siguiente configuración de tablero resuelto:

```
1  {
2    "tablero": [
3      [5, 3, 4, 6, 7, 8, 9, 1, 2],
4      [6, 7, 2, 1, 9, 5, 3, 4, 8],
5      [1, 9, 8, 3, 4, 2, 5, 6, 7],
6      [8, 5, 9, 7, 6, 1, 4, 2, 3],
```

```

7      [4, 2, 6, 8, 5, 3, 7, 9, 1] ,
8      [7, 1, 3, 9, 2, 4, 8, 5, 6] ,
9      [9, 6, 1, 5, 3, 7, 2, 8, 4] ,
10     [2, 8, 7, 4, 1, 9, 6, 3, 5] ,
11     [3, 4, 5, 2, 8, 6, 1, 7, 9]
12     ]
13     }

```

La señal de entrada `tablero[9][9]` representa la solución dada por el usuario, donde cada posición acepta valores comprendidos entre el 1 y el 9. Por otro lado, para la señal de salida `salida`, el valor 1 indica que el tablero es válido al cumplir todas las reglas del juego, mientras que el valor 0 indica que se ha incumplido al menos una regla y la solución propuesta es no es correcta.

De esta manera, acorde a esta asignación de valores, la configuración introducida representa un sudoku resuelto correctamente. Cualquier modificación sobre esta distribución que lleve a duplicar la presencia de números en filas, columnas o subtableros resultaría en `salida = 0`. Por tanto, el resultado para la actual configuración debería ser `salida = 1`.

Tras aplicar dichos valores a las señales de entrada y estudiar el `witness.json`, el valor de la señal de salida es 1 para ambas implementaciones.

Del mismo modo, se modificaron los valores de la matriz `tablero` en el archivo de entrada para comprobar que la salida es igual a 0 al utilizar configuraciones que rompen las reglas, así como el comportamiento con otros tableros que sí las cumplen, concluyendo así que la lógica es correcta para las distintas situaciones posibles.

Por otro lado, en cuanto al número de restricciones lineales y no lineales generadas por ambos códigos.

Para ello, evaluaremos de nuevo el comportamiento del circuito bajo los niveles de optimización -O0 y -O2, obteniendo los siguientes resultados:

Implementación	Número de restricciones O0	Número de restricciones O2
Sudoku sin <code>autocomplete</code>	No lineales: 5102 Lineales: 10506	No lineales: 5102 Lineales: 0
Sudoku con <code>autocomplete</code>	No lineales: 9865 Lineales: 1008	No lineales: 9865 Lineales: 0

Tras realizar estas compilaciones, se observa que, en ambos niveles de optimización, la implementación sin `autocomplete` es más eficiente que la versión con `autocomplete`, ya que genera un menor número de restricciones no lineales.

7.2. Aplicación de la generación automática sobre circomlib y comparación de constraints

Durante el capítulo anterior, para verificar el determinismo y la equivalencia de los operadores implementados, se realizaron comparaciones con las funciones de la librería oficial `circomlib`.

De esta forma, tras el análisis de equivalencia entre nuestros operadores y los propios de `circomlib`, se obtuvieron los siguientes resultados.

Cabe destacar que, al compilar bajo el nivel máximo de optimización, el número de restricciones lineales se reduce por completo a cero en todos los casos, por lo que la siguiente tabla se centra en evaluar el impacto sobre las restricciones no lineales al emplear este nivel de optimización:

Operador Individual	Restricciones No Lineales Sin autocomplete (O2)	Restricciones No Lineales Con autocomplete (O2)
Operadores Aritméticos		
Suma (+)	0	0
Resta (-)	0	0
Multiplicación (*)	1	1
División (/)	38	11
Módulo (%)	0	20
Operadores Lógicos y de Bits		
AND lógico (&&)	1	3
OR lógico ()	1	3
NOT lógico (!)	0	1
AND binario (&)	3	3
OR binario ()	3	3
Operadores de Comparación		
Igualdad (==)	2	2

Desigualdad ($\neq$)	2	2
Menor que ($<$)	2	4
Mayor que ($>$)	2	4
Menor o Igual que ($\leq$)	2	4
Mayor o Igual que ($\geq$)	2	4
Operadores de Desplazamiento		
Desplazamiento izquierda ($\ll$)	1	1
Desplazamiento derecha ($\gg$)	1	1

Conclusiones y Trabajo Futuro

8.1. Conclusiones

El propósito principal de este trabajo se ha alcanzado con éxito, logrando el objetivo de dotar al compilador de **Circom** de una mayor facilidad en cuanto a su uso, ofreciendo más opciones a la hora tanto de programar como de aprender el lenguaje.

A lo largo del proyecto, se ha conseguido automatizar la generación de restricciones para un gran cantidad de operadores y expresiones que antes de la incorporación de la nueva funcionalidad requerían una implementación manual tediosa y un proceso de adaptación largo, sobre todo en el primer contacto con Circom.

En términos de implementación, se han alcanzado con éxito los siguientes objetivos:

- **Simplificación en la implementación:** Gracias a la automatización de las restricciones, se reduce el volumen de código escrito a mano por el programador. Esto permite que el desarrollador construya circuitos complejos de forma mucho más rápida y con menos errores.
- **Aprendizaje y validación:** Al generar restricciones que siempre son correctas (ya que han sido verificadas), el programador puede usar esta herramienta para comprobar si sus circuitos en Circom están bien contruidos. De esta forma, el sistema sirve como un referente, garantizando que el circuito se comporta exactamente como se espera.
- **Programación más natural:** Se elimina la barrera de tener que pensar en términos de sistemas de ecuaciones. El programador puede expresar la lógica deseada tal y como quiere, dejando que el sistema se encargue de la traducción, lo que hace que trabajar con Circom sea mucho más sencillo.

Un aspecto imprescindible para alcanzar los objetivos propuestos ha sido la verificación. Se ha comprobado que las reglas de traducción implementadas son deterministas y que los valores de salida corresponden exactamente con los de la lógica esperada para cada operador y expresión. Sin embargo, el proceso de verificación mediante *ZK-GENVER* ha expuesto la gran dificultad de los operadores aritméticos de división y módulo.

Aunque estas funciones fueron implementadas en el compilador y se pudo verificar que son deterministas y que sus valores de salida son los esperados, la comprobación

de equivalencia tuvo como resultado *timeout*, lo que impidió verificar formalmente su equivalencia con los códigos de referencia que fueron desarrollados haciendo uso de funciones propias de la `circomlib` [B.1.2].

Por otro lado, excluyendo la división y el módulo, todos los demás consiguieron ser verificados con éxito, demostrando su equivalencia respecto a los operadores implementados utilizando Circom sin la nueva funcionalidad y funciones previamente implementadas en la librería `circomlib`.

En conclusión, este trabajo demuestra que es posible trabajar sobre el compilador para conseguir un comportamiento que simule la lógica de cada operador y expresión deseada, reduciendo la aparición de errores al hacer que el programador no este en la obligación de implementar sus propios operadores y expresiones, haciendo que el desarrollo de circuitos sea más accesible para programadores que no son expertos en lenguajes destinados a la generación de estos.

8.2. Trabajo Futuro

Partiendo de los resultados obtenidos, se plantean las siguientes propuestas de desarrollo futuro:

- **Implementación de otras estructuras:** Extender la traducción automática a estructuras de datos más avanzadas, como pueden ser los accesos a arrays, lo que permitiría manejar datos de forma más flexible.
- **Reducción del número de restricciones de cada operador:** Revisar las reglas de traducción para intentar reducir el número de restricciones en cada operador. El objetivo es que el compilador sea lo más eficiente posible, generando circuitos más simples y fáciles de procesar.
- **Reducción del número de restricciones global:** Una vez construido el sistema final de restricciones, aplicar optimizaciones para evitar restricciones duplicadas por diferentes expresiones, así como explorar otras vías de optimización del sistema completo.
- **Mejora de la eficiencia en la verificación.:** Explorar otras técnicas de verificación formal u otros entornos con mayor capacidad para validar los operadores de división y módulo.
- **Integración de la `circomlib`:** Automatizar el uso de las plantillas estándar de la librería. La idea es que los operadores del lenguaje puedan invocar directamente estas funciones sin que el programador tenga que conectarlas manualmente.

Conclusions and Future Work

9.1. Conclusions

The main purpose of this work has been successfully achieved, fulfilling the objective of providing the **Circom** compiler with greater ease of use, offering more options both when programming and learning the language.

Throughout the project, the generation of constraints has been successfully automated for a wide range of functionalities that, until now, required a much more tedious manual implementation and adaptation process, especially during the initial contact with Circom.

In terms of implementation, the following objectives have been successfully achieved:

- **Implementation optimization:** Thanks to the automation of constraints, the volume of code written by hand is significantly reduced. This allows the developer to build complex circuits much faster and with fewer errors.
- **Learning and validation:** By generating constraints that are always correct, the programmer can use this tool to verify whether their own Circom circuits are well-constructed. In this way, the system serves as a reference, guaranteeing that the circuit behaves exactly as expected.
- **More natural programming:** The barrier of having to think in terms of systems of equations is removed. The programmer can express the desired logic exactly as imagined, letting the system handle the translation, which makes the workflow much more agile and fluid.

A critical aspect in achieving the proposed objectives has been verification. It has been confirmed that the implemented translation rules are deterministic and that the output values correspond exactly to the expected logic for each operator and expression. However, the verification process using *ZK-GENVER* exposed the significant difficulty posed by the arithmetic operators for division and modulo.

Although these functions were implemented in the compiler and verified to be deterministic with the expected output values, the equivalence check resulted in a *timeout*. This prevented formal verification of their equivalence with the reference codes developed using standard `circomlib` functions [B.1.2].

On the other hand, excluding division and modulo, all other operators were successfully verified, demonstrating their equivalence to operators implemented in Circom without the new functionality and functions previously implemented in the `circomlib` library.

In conclusion, this work demonstrates that it is possible to modify the compiler to simulate the desired logic for each operator and expression. This reduces the occurrence of errors by removing the requirement for programmers to implement their own operators and expressions, making circuit development more accessible to developers who are not experts in languages designed for circuit generation.

9.2. Future Work

Based on the results obtained, the following proposals for future development are presented:

- **Implementation of additional structures:** Extend automatic translation to more advanced data structures, such as array access, allowing for more flexible data handling.
- **Reduction of the number of constraints for each operator:** Review the translation rules to minimize the number of constraints applied to each operator. The goal is to make the compiler as efficient as possible, generating simpler and easier-to-process circuits.
- **Global restriction count reduction:** Once the final system of restrictions has been built, apply optimizations to avoid duplicate restrictions from different expressions, as well as explore other optimization pathways for the complete system.
- **Efficiency improvements in verification:** Explore alternative formal verification techniques or environments with greater capacity to validate division and modulo operators.
- **Integration of `circomlib`:** Automate the use of standard library templates. The idea is for language operators to directly invoke these functions without the programmer having to connect them manually.

Bibliografía

- [1] Wikipedia. (2026). *Prueba de conocimiento cero*. https://es.wikipedia.org/wiki/Prueba_de_conocimiento_cero.
- [2] Nervos Network. (2024). *Pruebas de conocimiento cero (zero knowledge proofs)*. [https://www.nervos.org/es/knowledge-base/zero_knowledge_proofs_\(explainCKBot\)](https://www.nervos.org/es/knowledge-base/zero_knowledge_proofs_(explainCKBot)).
- [3] Binance Academy. (2024). *ZK-SNARKs y ZK-STARKs explicados*. <https://www.binance.com/es/academy/articles/zk-snarks-and-zk-starks-explained>.
- [4] Rodríguez Nuñez, C. (2024). *Propiedades de corrección y seguridad en tecnologías blockchain*.
- [5] Buterin, V. (2016). *Quadratic Arithmetic Programs: from Zero to Hero*. <https://vitalik.eth.limo/general/2016/12/10/qap.html>. Accedido: 04-04-2026.
- [6] Wikipedia. (s. f.). *Cuerpo finito*. https://es.wikipedia.org/wiki/Cuerpo_finito.
- [7] Bellés-Muñoz, M., Isabel, M., Muñoz-Tapia, J. L., Rubio, A. y Baylina, J. (2023). CIRCOM: A Circuit Description Language for Building Zero-Knowledge Applications. *IEEE Software/Journal*. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10002421>.
- [8] BattleZips Team. (2022). *BattleZips-Circom: Zero-knowledge Battleship engine*. <https://github.com/BattleZips/BattleZips-Circom>.
- [9] Dark Forest Team. (2021). *Dark Forest Circuits*. <https://github.com/darkforest-eth/circuits>.
- [10] Iden3. (2023). *Circuits/multiplexer.circom*. <https://github.com/iden3/circomlib/blob/master/circuits/multiplexer.circom>.
- [11] Martínez, L. (2026). *Implementación de juegos simples con Circom*. <https://github.com/luciamarst/circom/tree/Juegos>.
- [12] Studocu. (s. f.). *Teorema de la división entera*. Universitat Politècnica de València. <https://www.studocu.com/es/>

document/universitat-politecnica-de-valencia/matematicas/
teorema-de-la-division-entera/59857285.

- [13] Iden3. (s. f.). *Circuits/comparators.circom*. <https://github.com/iden3/circomlib/blob/master/circuits/comparators.circom>
- [14] Iden3. (s. f.). *Circuits/bitify.circom*. <https://github.com/iden3/circomlib/blob/master/circuits/bitify.circom>
- [15] Costa Group. (2024). *ZK-GENVER: A tool for zero-knowledge circuit generation and verification*. <https://github.com/costa-group/ZK-GENVER>.
- [16] Iden3. (2023). *circomlib: Library of basic circuits for Circom*. <https://github.com/iden3/circomlib>.
- [17] Iden3. (s. f.). *Circuits/gates.circom*. <https://github.com/iden3/circomlib/blob/master/circuits/gates.circom>.
- [18] Rust Foundation. (2025). *Rust: Un lenguaje que empodera a todos para crear software fiable y eficiente*. <https://www.rust-lang.org/es/>.
- [19] Wikipedia. (2024). *Árbol de sintaxis abstracta*. Wikipedia, La enciclopedia libre. https://es.wikipedia.org/wiki/%C3%81rbol_de_sintaxis_abstracta.
- [20] Martínez, L. (2025). *Circom autocompile: zkSnark circuit compiler*. https://github.com/luciamarst/circom_autocomplete.
- [21] Geeknetic. (2023). *Cómo convertir binario en decimal paso a paso*. Geeknetic. <https://www.geeknetic.es/Guia/2667/Como-convertir-binario-en-decimal-paso-a-paso.html>.

CAPÍTULO A

Problemática de los números negativos

Los número negativos no existen en Circom. El motivo de que estos números no existan es que trabajamos sobre un cuerpo finito $\mathbb{F}_p$ definido por un número primo cuyo tamaño es 254 bits. En este sistema, cualquier operación se realiza módulo p , por lo que el rango de elementos válidos es estrictamente $[0, p - 1]$.

En consecuencia, un número negativo $-x$ se interpreta mediante su equivalente positivo en el cuerpo, calculado como $p - x$. Por ejemplo, el valor -1 se representa como $p - 1$, un número extremadamente grande cercano al límite superior del campo.

Estos números excesivamente grandes pueden provocar comportamientos no esperados en operadores como $<, \leq, >$ y $\geq$, ya que al realizar las comparaciones entre números negativos su conversión a estos números positivos grandes puede tener como resultado una comparación errónea.

Verificación de operadores

B.1. Operadores Aritméticos

B.1.1. División

El operador división, como se explico en la **Sección 4.1.1.1**, se basa en el teorema de la división para poder simular el comportamiento de este operador.

Por tanto, al no existir una función que imite este comportamiento en la `circomlib`, se implementó una haciendo uso de las funciones `IsZero` y `LessThan(n)` que la librería ofrece.

Estas funciones son usadas para asegurar que las restricciones $b \neq 0$ y $0 \leq r < |b|$ se cumplen. De esta forma, el resultado del código sería el siguiente:

Ejemplo B.1: Creación de división usando funciones de `circomlib`

```

1  template circomlibDivision(n){
2      signal input a;
3      signal input b;
4      signal output salida; //Cociente
5
6      // Asegurar que no son numeros negativos
7      component n2b_a = Num2Bits(2);
8      n2b_a.in <== a;
9
10     component n2b_b = Num2Bits(2);
11     n2b_b.in <== b;
12
13     signal q<--a\b; //Cociente
14     signal r<--a%b; //Resto
15
16     // Comprobamos que b!=0
17     component is_zero = IsZero();
18     is_zero.in<==b;
19     is_zero.out == 0;
20
21     // Comprobamos que 0<=r<|b|; 0 <= r no hace falta porque al estar en
22     // campo finito se entiende
23     // r < |b|
24     component lt = LessThan(2);
25     lt.in[0] <== r;
26     lt.in[1] <== b;
27     lt.out == 1;
28
29     // Una vez hecho eso, comprobamos que se cumple a == b*q+r
30     a == b*q +r;
31     salida <==q;
32 }
33 component main = circomlibDivision();

```

Observación: El bloque de código expuesto es muy similar al pseudocódigo presentado en la **Sección 4.1.1.1**.

B.1.2. Módulo

El operador módulo seguiría una lógica similar a la expuesta en la división, con la única diferencia que la salida sería el resto en lugar del cociente, ya que en este caso es el valor esperado para esta operación.

B.2. Operadores lógicos

B.2.1. AND binario

La librería `circomlib` no contiene una función explícita dedicada al operador AND binario, por lo que para verificar la equivalencia entre ambos códigos, se ha implementado una función utilizando otras funciones pertenecientes a la librería para simular este comportamiento.

Para dicha implementación serán necesarias las funciones `Num2Bits(n)` y `Bits2Num(n)` de la librería, ubicadas en el archivo `bitify.circom`.

Con la ayuda de estas funciones se ha podido implementar la siguiente función para el operador AND bit a bit.

Ejemplo B.2: Creación de AND bit a bit usando funciones de `circomlib`

```
1  template BitwiseAnd(n) {
2      signal input a;
3      signal input b;
4      signal output out;
5
6      // Pasamos a
7      component n2b_a = Num2Bits(n);
8      component n2b_b = Num2Bits(n);
9      component b2n = Bits2Num(n);
10
11     n2b_a.in <== a;
12     n2b_b.in <== b;
13
14     // Aplicamos el && a cada par de bits con una multiplicacion
15     // Metemos el resultado en bits2num directamente, el resultado devuelto
16     // es decimal
17     for (var i = 0; i < n; i++) {
18         b2n.in[i] <== n2b_a.out[i] * n2b_b.out[i];
19     }
20     out <== b2n.out;
21 }
22
23 component main = BitwiseAnd(1);
```

De esta forma, comparamos el código implementado con el de la nueva funcionalidad.

Tras la comprobación determinista de ambos códigos mediante el análisis de restricciones R1CS, se procedió a la verificación de equivalencia.

Cuadro B.1: Comparativa de la operación OR bit a bit (1 bit) entre la implementación estándar y el uso de autocomplete.

Implementación circomlib (Manual)	Implementación Autocomplete
<pre> template BitwiseOr(n) { signal input a, b; signal output out; component n2b_a = Num2Bits(n); component n2b_b = Num2Bits(n); component b2n = Bits2Num(n); n2b_a.in <== a; n2b_b.in <== b; for (var i = 0; i < n; i++) { b2n.in[i] <== n2b_a.out[i] * n2b_b.out[i]; } out <== b2n.out; } </pre>	<pre> template orb() autocomplete { signal input a; signal input b; signal output c; // Operacion bitwise OR // inferida automaticamente c <-- a &_(1) b; } </pre>

B.2.2. OR binario

El operador OR binario presenta el mismo problema que AND, ya que en la `circomlib` no hay una función explícita dedicada a estos operadores. Para solucionarlo, seguimos los mismos pasos que para el operador AND bit a bit, por lo que implementamos una función similar para OR bit a bit.

De esta forma, los códigos implementados son prácticamente iguales:

Cuadro B.2: Comparativa de la operación OR bit a bit (1 bit) entre la implementación estándar y el uso de autocomplete.

Implementación circomlib (Manual)	Implementación Autocomplete
<pre> template BitwiseOr(n) { signal input a, b; signal output out; component n2b_a = Num2Bits(n); component n2b_b = Num2Bits(n); component b2n = Bits2Num(n); n2b_a.in <== a; n2b_b.in <== b; for (var i = 0; i < n; i++) { b2n.in[i] <== n2b_a.out[i] + n2b_b.out[i] - n2b_a.out[i] * n2b_b.out[i]; } out <== b2n.out; } </pre>	<pre> template orb() autocomplete { signal input a; signal input b; signal output c; // Operacion bitwise OR // inferida automaticamente c <-- a _(1) b; } </pre>

B.3. Operadores de desplazamiento

B.3.1. Desplazamiento a la izquierda

Para poder realizar la prueba de equivalencia se ha tenido que implementar un código en Circom base, ya que en la librería `circomlib` no existe ninguna función que simule explícitamente el comportamiento de estos operadores.

De esta forma, el código desarrollado para comprobar la equivalencia con el desplazamiento a la izquierda ha sido el siguiente:

```
1      include "bitify.circom";
2
3      template shiftL(n){
4          signal input a;
5          signal output c;
6
7          component n2b = Num2Bits(n);
8          n2b.in <== a;
9
10         signal shifted_bits[n];
11
12         shifted_bits[0] <== 0;
13         shifted_bits[1] <== 0;
14
15         shifted_bits[2] <== n2b.out[0];
16         shifted_bits[3] <== n2b.out[1];
17
18         //Convertir a decimal
19         component b2n = Bits2Num(n);
20
21         b2n.in[0] <== shifted_bits[0];
22         b2n.in[1] <== shifted_bits[1];
23         b2n.in[2] <== shifted_bits[2];
24         b2n.in[3] <== shifted_bits[3];
25
26
27         c <== b2n.out;
28     }
29
30     component main = shiftL(4);
```

Aunque en la librería no haya funciones explícitas dedicadas a los desplazamientos, para su implementación se han usado otras funciones como `Num2Bits()` y `Bits2Num()` para facilitar las conversiones numéricas requeridas por este operador.

Observando el ejemplo, puede observarse que se ha implementado para un tamaño de $n = 4$ y desplazamiento $k = 2$, ya que en el programa codificado con el uso de la nueva funcionalidad es el siguiente:

```
1      pragma circom 2.2.2;
2
3      template funcion() autocomplete{
4          signal input a;
5          signal output b;
6
7          b <-- a <<_(4) 2;
8      }
9
10     component main = funcion();
```

B.3.2. Desplazamiento a la derecha

De manera análoga, aplicamos el mismo patrón para el desplazamiento a la derecha, utilizando los mismos valores tanto para el tamaño como para el desplazamiento, obteniendo un código en Circom tal que así:

```
1      include "bitify.circom";
2
3      template shiftL(n) {
4          signal input a;
5          signal output c;
6
7          component n2b = Num2Bits(n);
8          n2b.in <== a;
9
10         signal shifted_bits[n];
11
12         for (var i = 0; i < 2 && i < n; i++) {
13             shifted_bits[i] <== 0;
14         }
15
16         for (var i = 2; i < n; i++) {
17             shifted_bits[i] <== n2b.out[i - 2];
18         }
19
20         component b2n = Bits2Num(n);
21         for (var i = 0; i < n; i++) {
22             b2n.in[i] <== shifted_bits[i];
23         }
24
25         c <== b2n.out;
26     }
27
28     component main = shiftL(4);
```

El código Circom de la nueva funcionalidad para este operador es muy similar al expuesto para el otro caso, ya que únicamente hay que cambiar el operador.

```
1      pragma circom 2.2.2;
2
3      template funcion() autocomplete{
4          signal input a;
5          signal output b;
6
7          b <-- a >>_(4) 2;
8      }
9
10     component main = funcion();
```
